Министерство науки и высшего образования РФ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Уфимский университет науки и технологий»

Кафедра: Высокопроизводительных вычислений и дифференциальных уравнений

ИМИТАЦИОННОЕ МОДЕЛИРОВАНИЕ СЛОЖНЫХ СИСТЕМ С ПОМОЩЬЮ КЛЕТОЧНЫХ АВТОМАТОВ

ОТЧЕТ

к лабораторной работе №5 по дисциплине

«Математическое моделирование»

Группа МКН-418Б	Фамилия И. О.	Подпись	Дата	Оценка
Студент	Халитова А.А.			
Принял	Феоктистов Б.А.			

Уфа 2026

Содержание

1 Введение **3**

2 Клеточный автомат «Жизнь» **4**
2.1 Постановка задачи 4
2.2 Ход работы 4
2.2.1 B2/S012 4
2.2.2 B3/S23 9

3 Клеточный автомат «Нейронная сеть» **13**
3.1 Постановка задачи 13
3.2 Ход работы 14

4 Клеточный автомат «Организмы - питательная среда» . **18**
4.1 Постановка задачи 18
4.2 Ход работы 20

5 Модифицированный клеточный автомат «Организмы - питательная среда» **22**
5.1 Постановка задачи 22
5.2 Ход работы 23

6 Вывод **24**

7 Листинг **25**

1 Введение

Цель работы: получить навык имитационного моделирования сложных динамических систем с использованием клеточных автоматов на примере моделей биологических систем.

2 Клеточный автомат «Жизнь»

2.1 Постановка задачи

Выполнить программную реализацию клеточного автомата, функционирующего в соответствии со следующими правилами:

1. клетка может находиться в двух состояниях - пассивном и активном;
2. в качестве окрестности рассматривается восемь соседних клеток;
3. если в окрестности пассивной клетки две активных, то данная клетка также становится активной («рождается»);
4. если в окрестности активной клетки три или более активных клеток, то она становится пассивной («умирает»).

Реализовать алгоритм на клеточном пространстве 32×32 ячеек. Начальное распределение активных и пассивных клеток – случайное, подчиняющееся равномерному закону распределения. Также подобрать начальные распределения, соответствующие стационарным и циклическим структурам (по три примера каждой структуры).

2.2 Ход работы

На языке программирования C++ была написана программа, реализующая клеточный автомат. Начальное распределение активных и пассивных клеток – случайное, подчиняется равномерному закону распределения.

2.2.1 B2/S012

Эксперименты проводились на клеточном автомате "Жизнь"с набором правил B2/S012 (рождение при двух соседях, выживание 0, 1 или 2 соседях).

На рисунке 1 показано начальное положение системы. Красным цветом отмечены активные клетки, черным – пассивные.

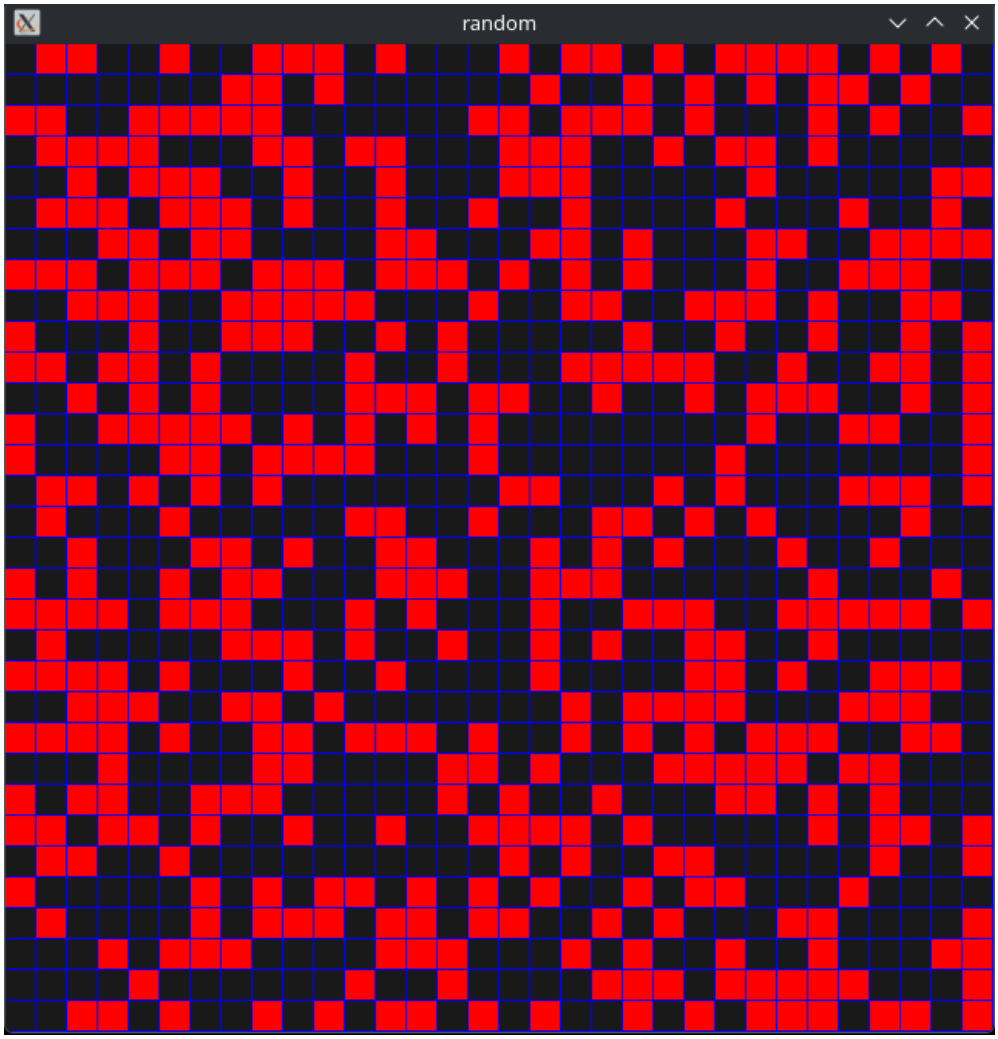

Рисунок 1 — Начальное случайное распределение активных клеток

Состояние системы после 5 тактов показано на рисунке 2.

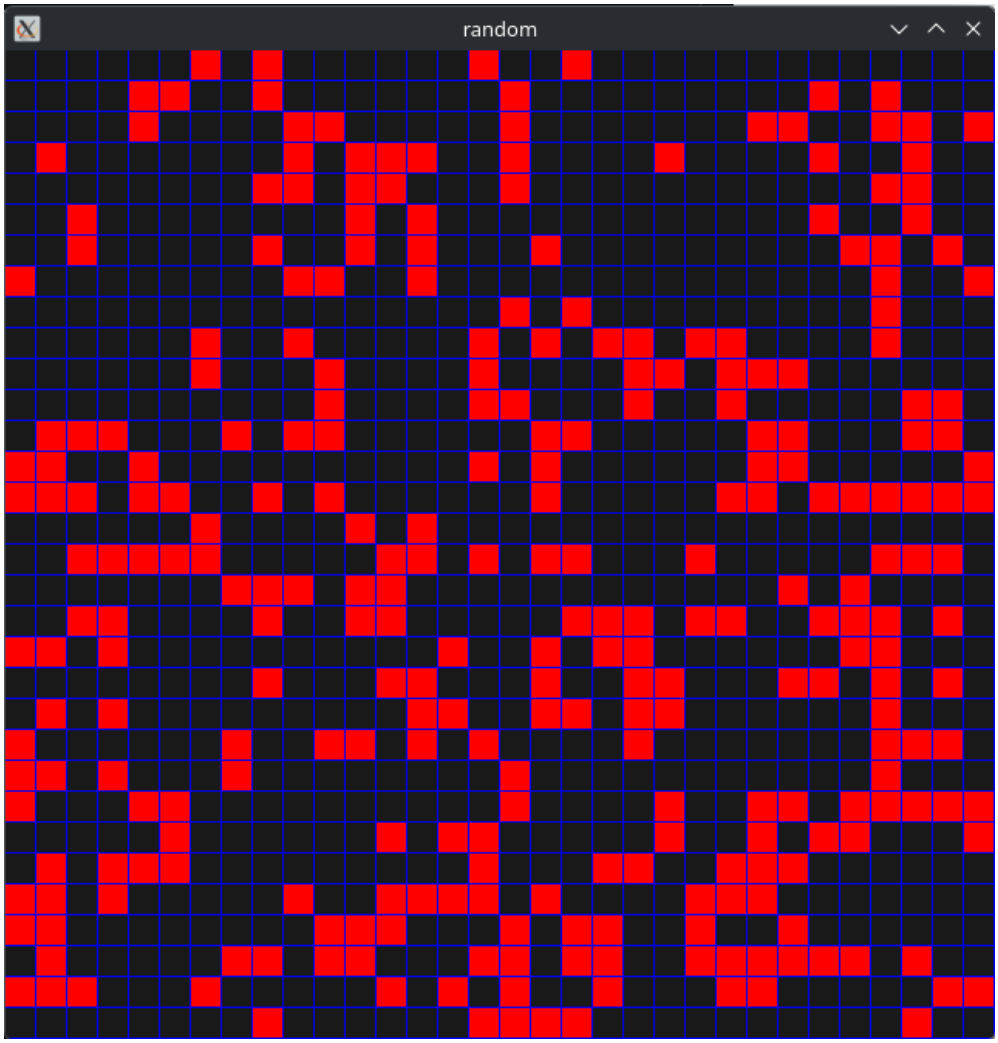

Рисунок 2 — Система после 5 итераций

Из-за мягких условий выживания система не склонна формированию сложных изолированных структур.

Начальные распределения, соответствующие стационарным структурам показаны на рисунке 3.

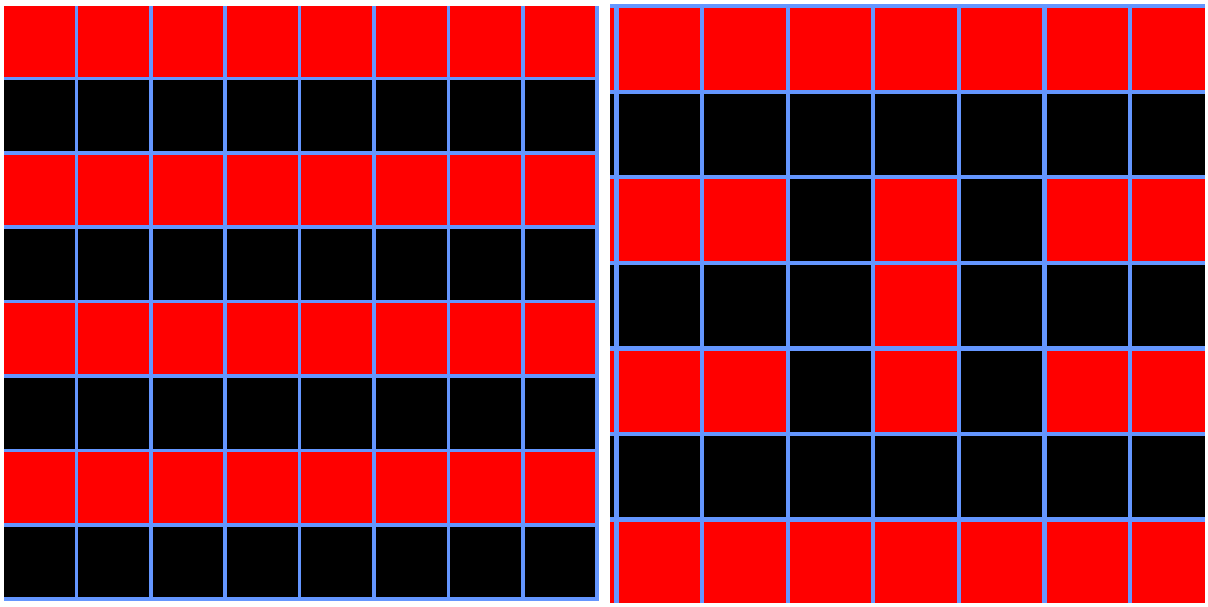

Рисунок 3 — Структура в виде горизонтальных линий

Начальные распределения, соответствующие циклическим структурам показаны на рисунках 4 - 8.

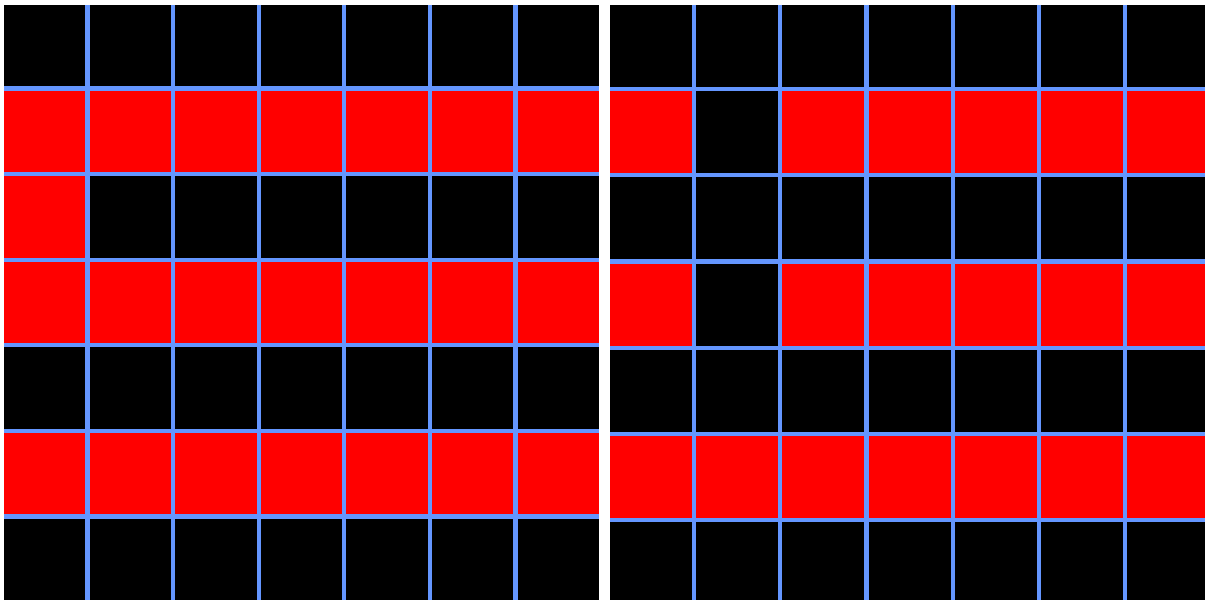

Рисунок 4 — Циклическая структура с периодом 2

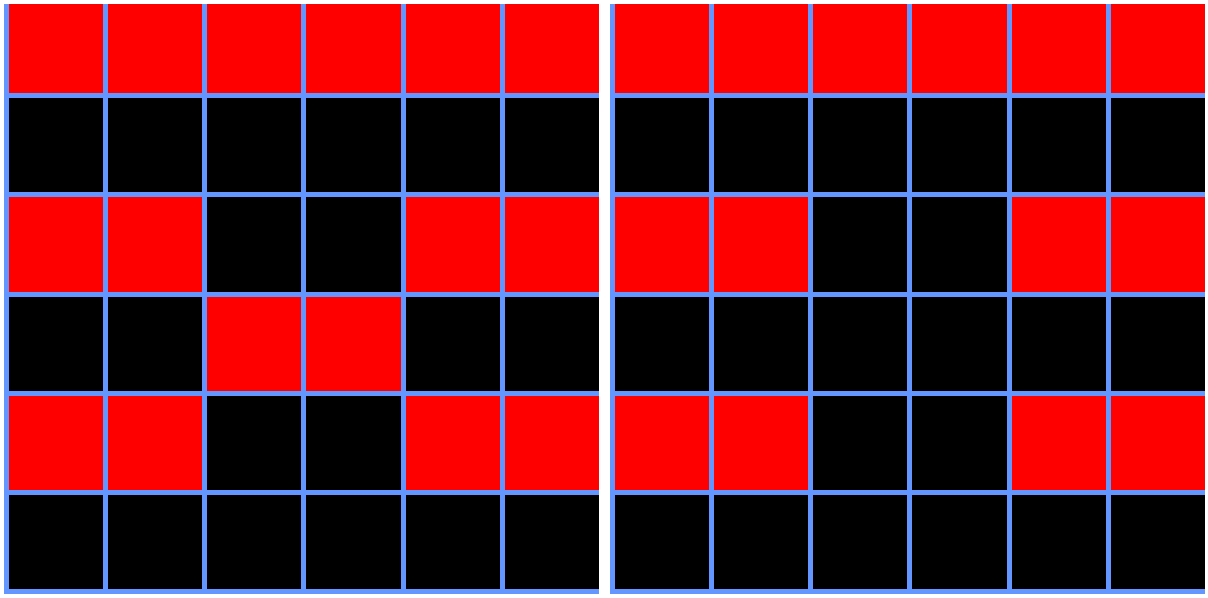

Рисунок 5 — Циклическая структура с периодом 2

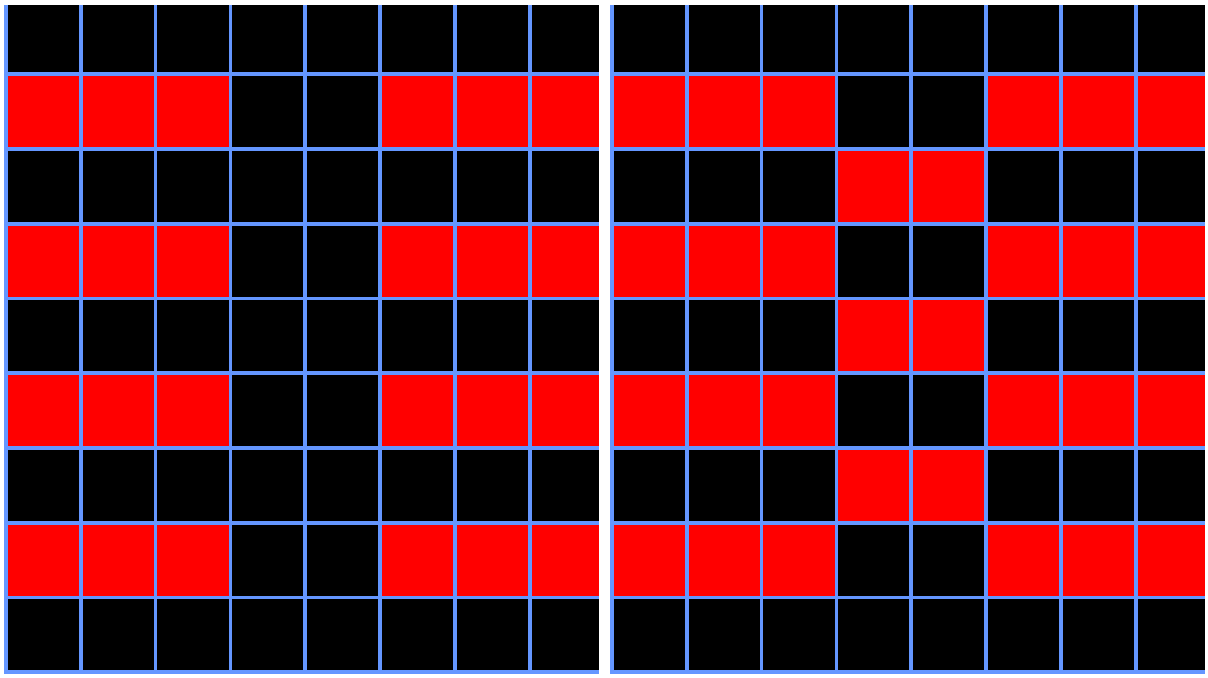

Рисунок 6 — Циклическая структура с периодом 6

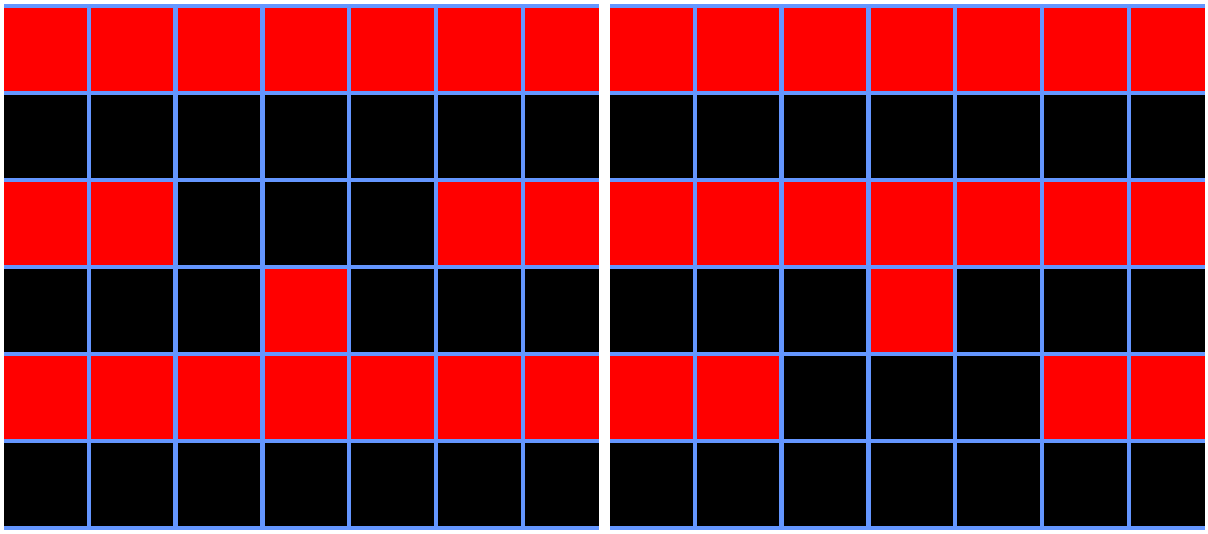

Рисунок 7 — Циклическая структура с периодом 4

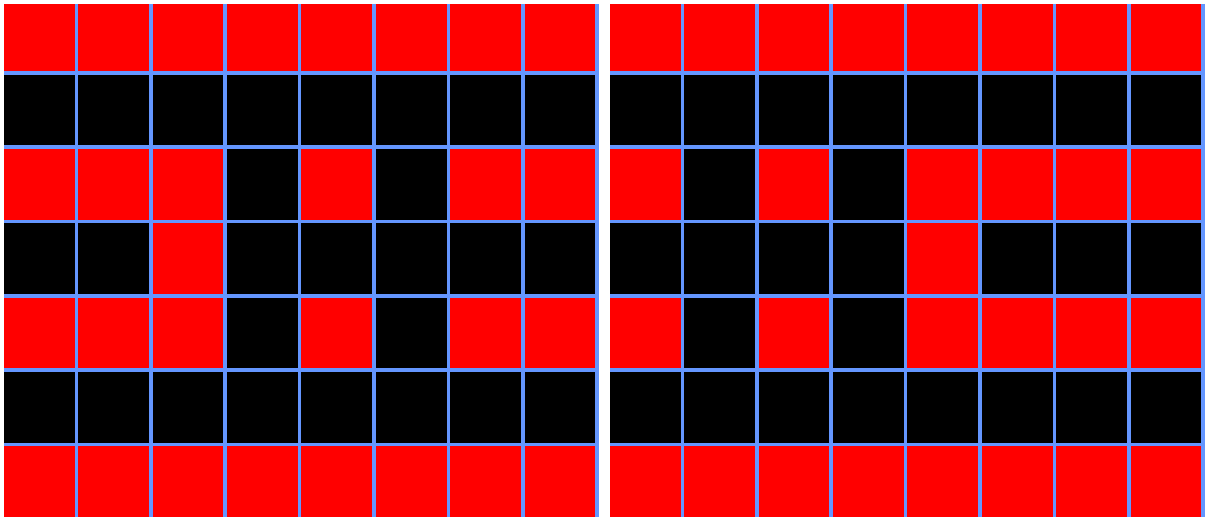

Рисунок 8 — Циклическая структура с периодом 2

2.2.2 B3/S23

Эксперименты проводились на классическом клеточном автомате "Жизнь"с набором правил B3/S23 (рождение при трех соседях, выживание 2 или 3 соседях).

На рисунке 9 показано начальное положение системы. Красным цветом отмечены активные клетки, черным – пассивные.

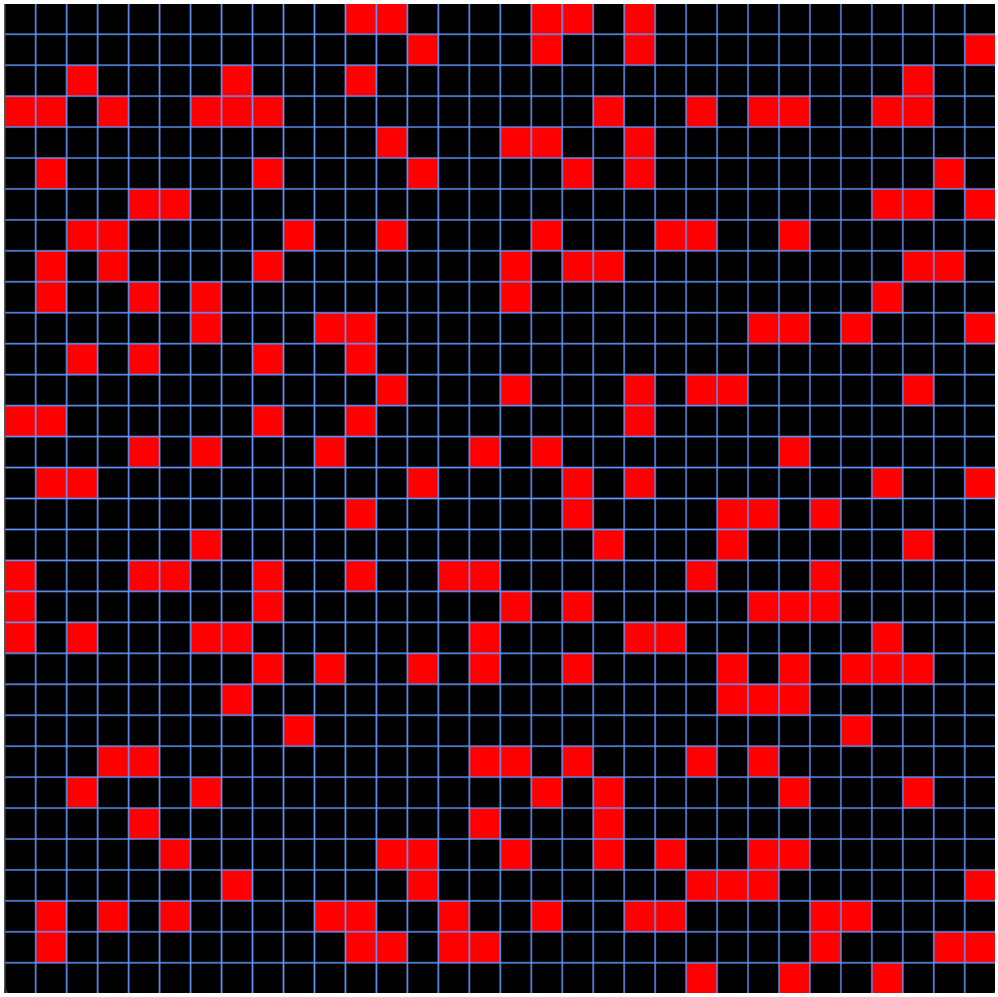

Рисунок 9 — Начальное случайное распределение активных клеток

Состояние системы после 5 тактов показано на рисунке 10.

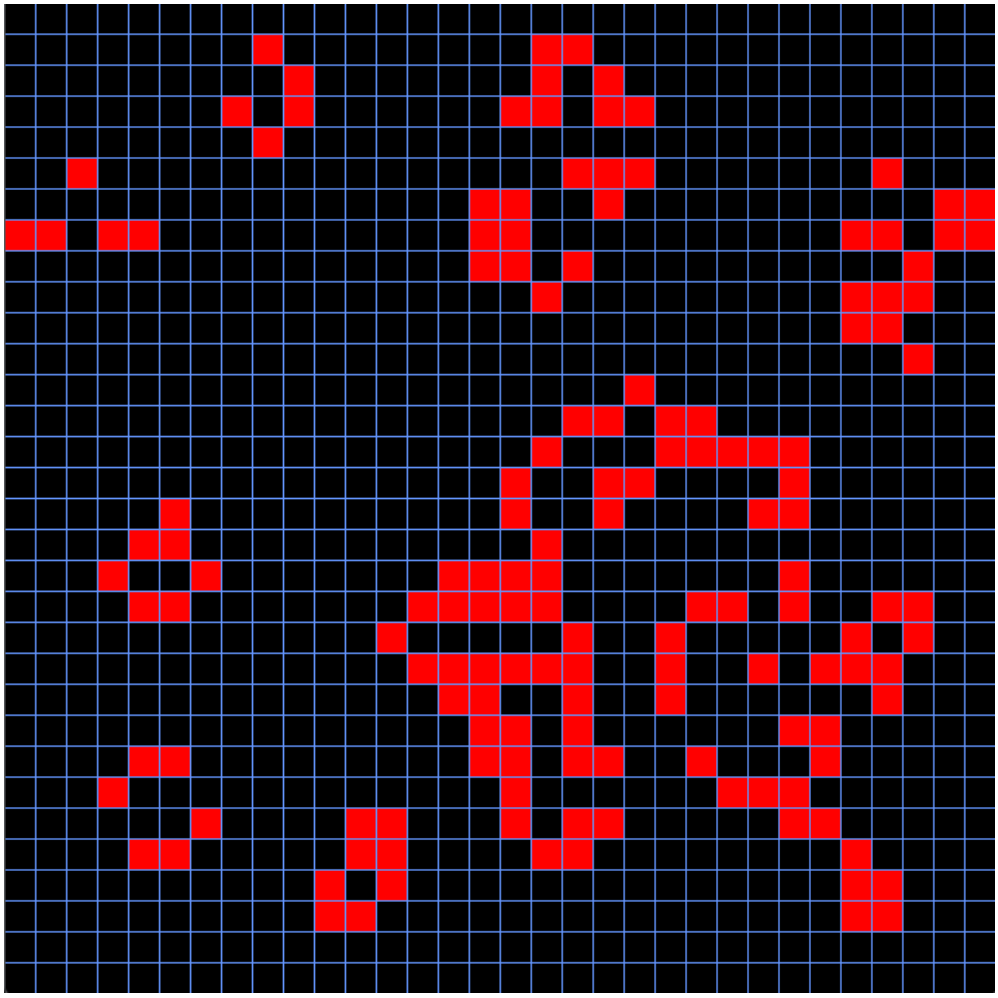

Рисунок 10 — Состояние системы после 5 тактов

Из-за крайне жестких условий выживания система склонна формированию сложных изолированных структур.

Начальные распределения, соответствующие стационарным структурам показаны на рисунке 11.

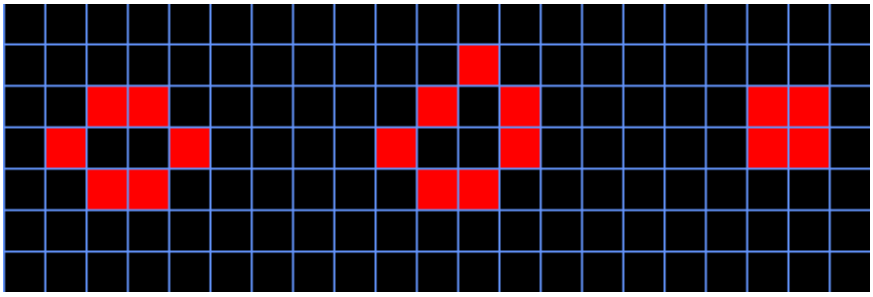

Рисунок 11 — Стационарные структуры

Начальные распределения, соответствующие циклическим структурам показаны на рисунках 12 - 8.

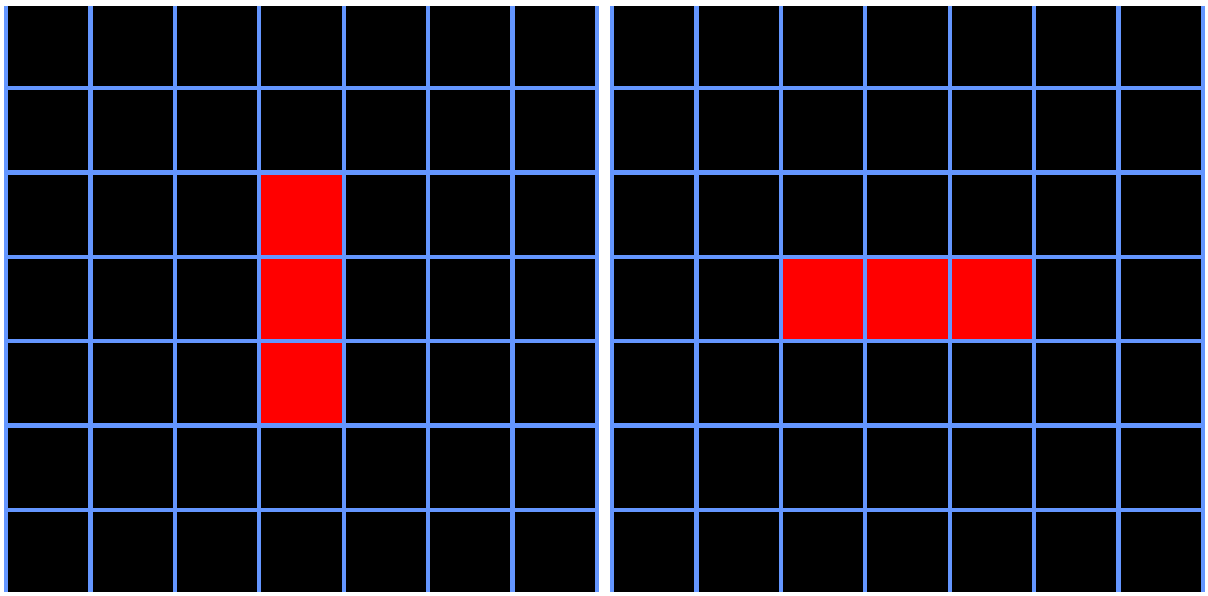

Рисунок 12 — Циклическая структура с периодом 2

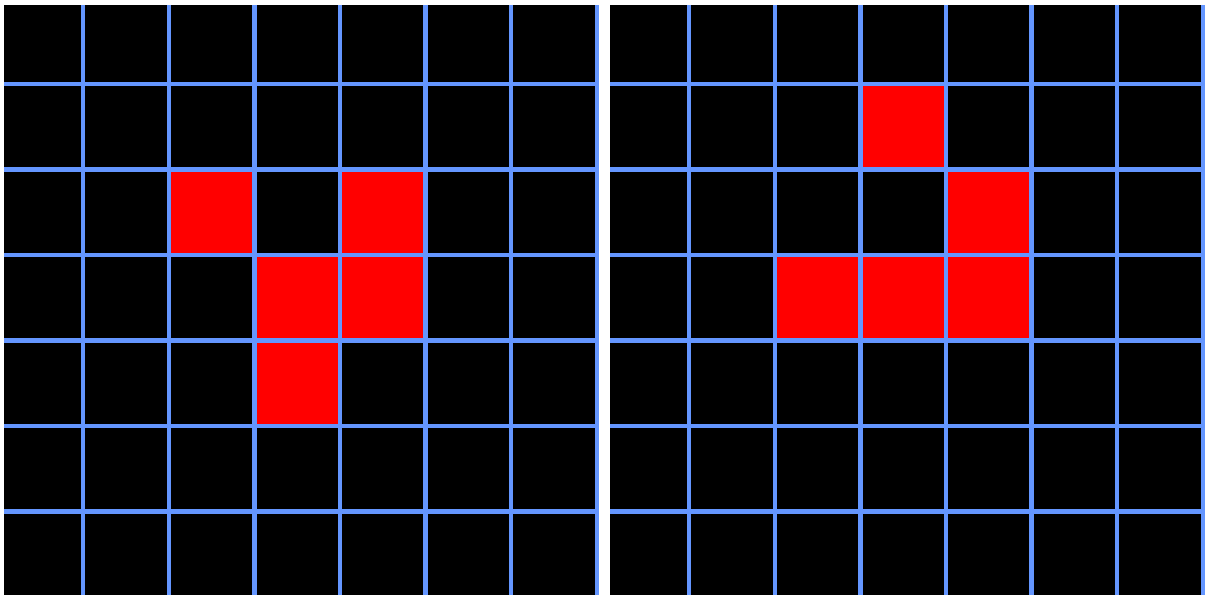

Рисунок 13 — Циклическая структура с периодом 4

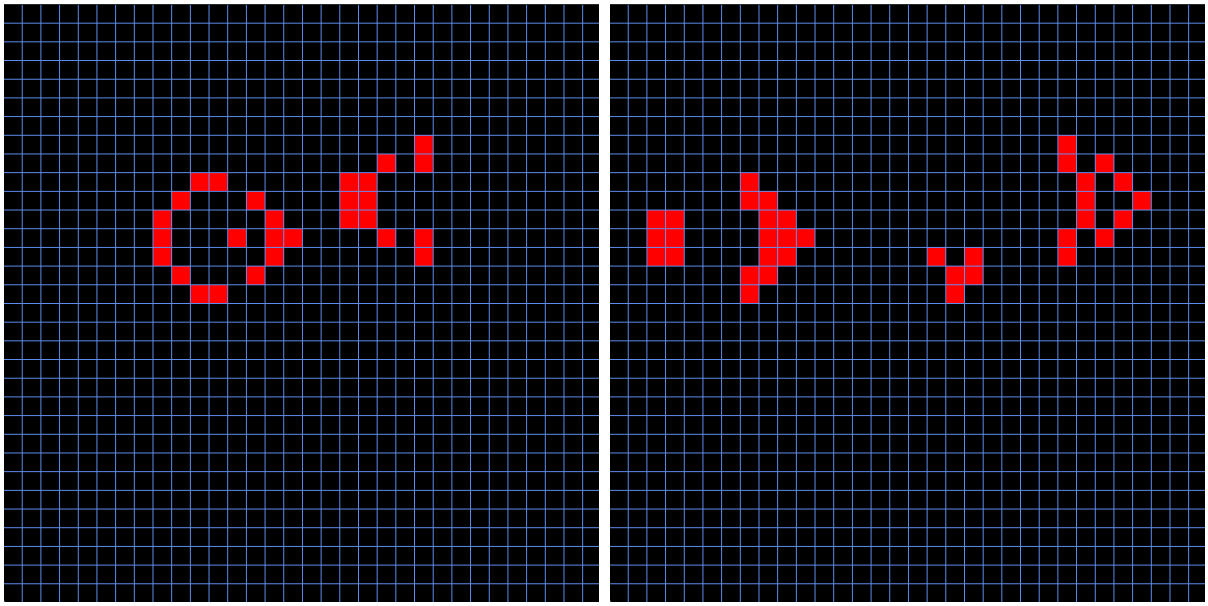

Рисунок 14 — Циклическая структура пушка Госпера с периодом 30

3 Клеточный автомат «Нейронная сеть»

3.1 Постановка задачи

Данный автомат имитирует явления в однородной двумерной нейронной сети, состоящей из возбудимых элементов и функционирует по следующим правилам:

1. клетка может находиться в трех состояниях: покоя, активном и состоянии восстановления;

2. в качестве окрестности рассматриваются восемь соседних клеток;

3. переход в состояние активности зависит от некоторого параметра, называемого уровнем активатора. В возбужденном состоянии клетки уровень активатора равен 1. В других состояниях он распадается на $A\%$ такт;

4. если клетка была в покое и общее количество активатора в восьми соседних и в данной клетке превысило порог активации P, то клетка возбуждается на T тактов;

5. через T тактов возбужденная клетка переходит в состояние восстановления на B тактов, а затем в переходит в состояние покоя.

Реализовать алгоритм при следующих параметрах: клеточное пространство 256×256 ячеек, $A = 30\%$, $P = 3$, $T = 5$, $B = 8$. Начальное распределение состояния клеток задано плоским фронтом. Также имеется периодический источник возбуждения (3×3 клетки) с периодом 15 тактов. Выявить характер взаимодействия между собой различных фронтов возбуждения.

3.2 Ход работы

Клетка может находиться в трех состояниях: покоя – зеленый цвет, активном – красный цвет и состоянии восстановления – синий цвет. Начальное распределение состояния клеток (рисунок 15) задано плоским фронтом в виде вертикальной прямой в правой части поля и точечный источник в левой.

Эксперименты проводились на сетке размером 128×128 клеток для лучшей визуализации.

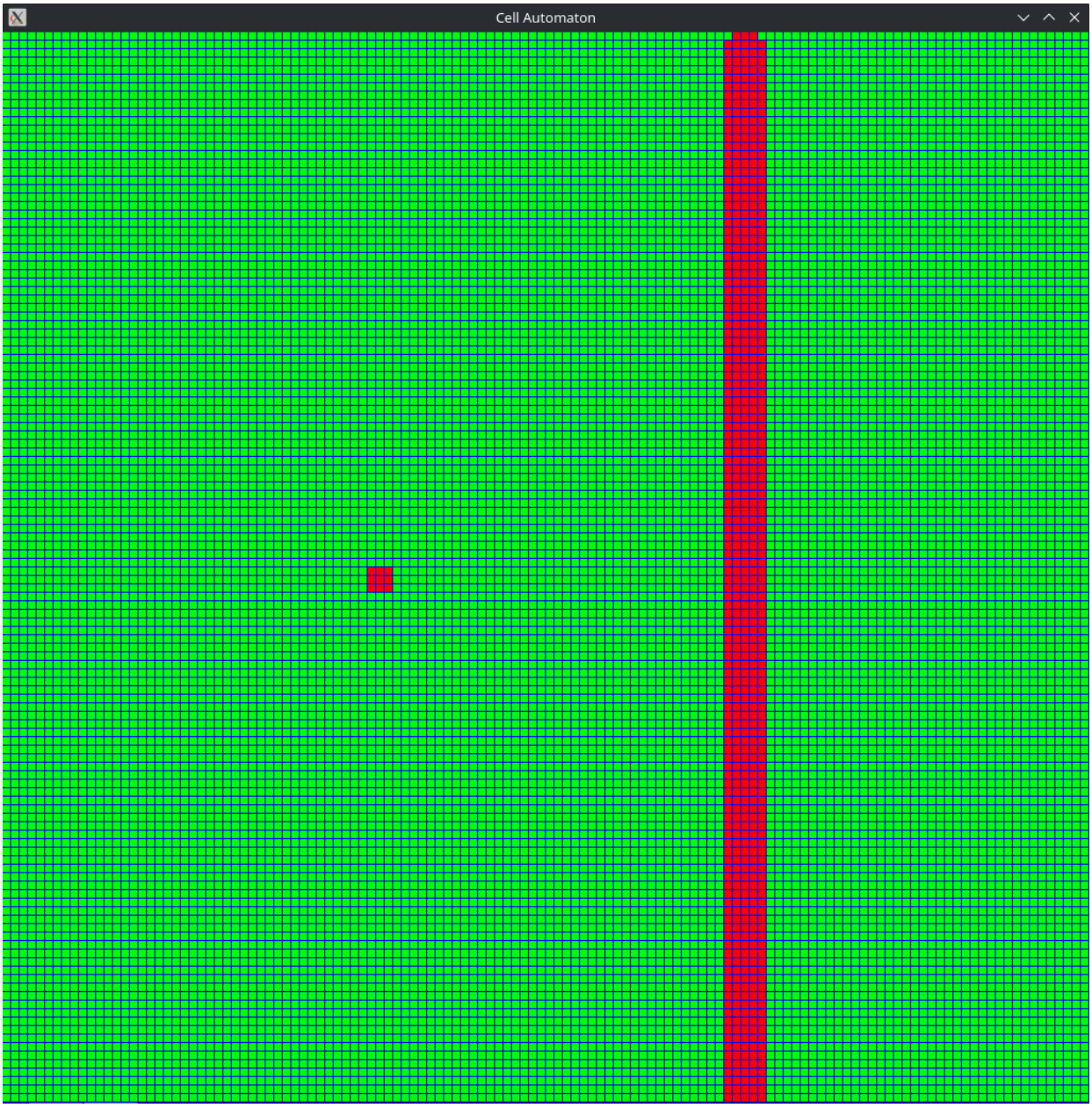

Рисунок 15 — Начальное состояние системы

В процессе эволюции системы (рисунок 16) плоский фронт распространяется и формирует равномерную волну активации, а точечный источник генерирует волны, расходящиеся во все стороны. За активной фазой (красный цвет) следует фаза восстановления (синий цвет).

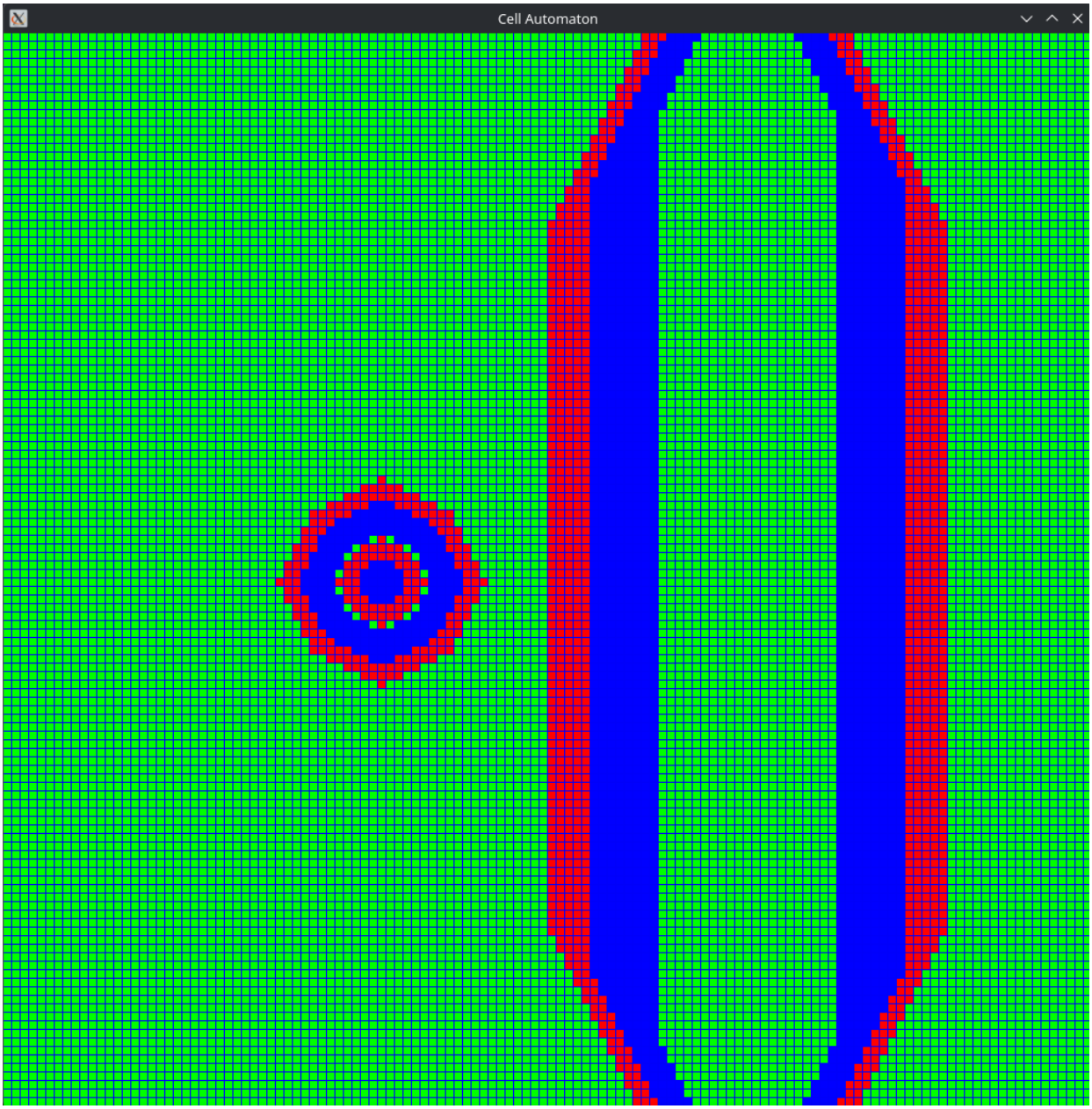

Рисунок 16 — Движущийся фронт и источник возбуждения

При столкновении волн (рисунок 17) происходит их взаимное гашение. Клетки, находящиеся в фазе восстановления (синий цвет), не могут быть активированы повторно, что приводит к аннигиляции (исчезновению) волн в области их встречи.

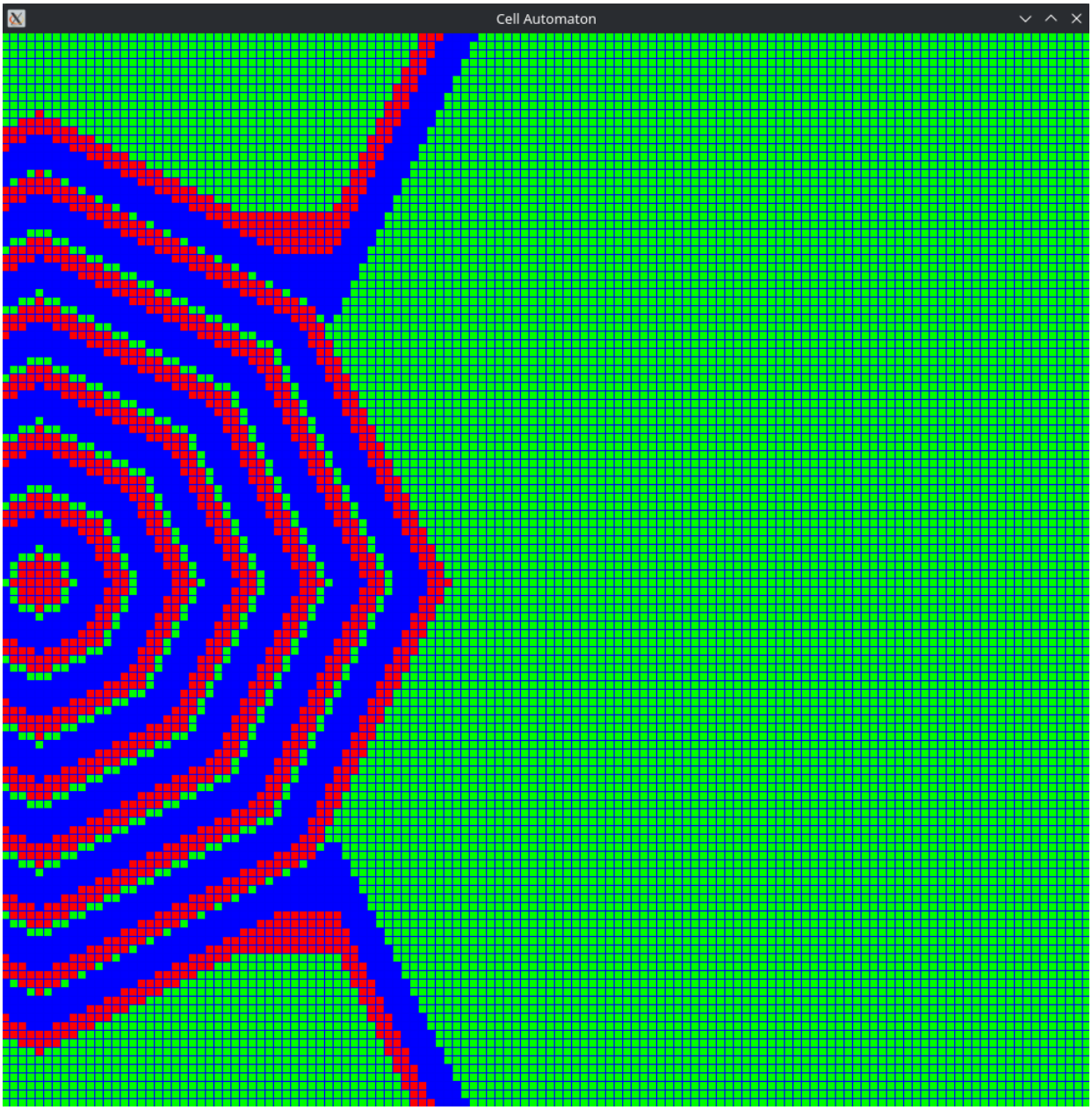

Рисунок 17 — Взаимное гашение встречных фронтов возбуждения

На рисунке 18 показан эффект слияния волновых фронтов, распространяющихся в одном направлении. В результате образуется единый волновой фронт, который распространяется по полю до встречи с препятствием.

Рисунок 18 — Соединение и усиление волновых фронтов

4 Клеточный автомат «Организмы - питательная среда»

4.1 Постановка задачи

Клеточный автомат моделирует взаимодействие одноклеточных организмов с питательной средой и функционирует по следующим правилам:

1. клеточное пространство образует поле $N \times N$ клеток;
2. окрестность клетки составляют восемь соседних клеток;

3. каждой клетке соответствует значение P степени питательности раствора (энергоемкости), которое может изменяться от 0 до $P_{\max}$;

4. прирост ΔP питательности (энергоемкости) раствора клетки за такт времени выполняется следующим образом: $\Delta P = 0$ при $P = P_{\max}$ и $\Delta P = r$ при $P < P_{\max}$, где r – скорость прироста питательности;

5. общий запас энергии питательного раствора определяется суммарной питательностью (энергией) всех клеток и не может быть более $N^2 P_{\max}$;

6. клетка может быть свободной или содержать не более одного одноклеточного или другого живого организма;

7. отдельная особь одноклеточного черпает энергию из питательного раствора клетки, в которой она находится, снижая его питательность и повышая свой запас энергии на Δp за такт;

8. максимально возможное количество энергии, запасаемое одноклеточным, не превышает p_1;

9. на свои нужды отдельная особь затрачивает Δe энергии за такт;

10. особь всегда старается перейти на соседнюю свободную клетку, выбирая направление перехода случайным образом;

11. время жизни отдельной особи составляет L тактов;

12. если время жизни особи превысило продолжительность жизни для данных организмов или запас энергии снизился до нуля, то особь умирает;

13. начиная с возраста T тактов особь считается зрелой и может производить себе подобных, затрачивая Δr энергии при каждом делении дополнительно. При этом старая особь переходит на свободную соседнюю клетку, а новая остается в старой. Если свободных клеток в окрестности нет, то деления не происходит;

14. начальное распределение особей по клеточному пространству подчинено равномерному закону распределения. Начальное число особей составляет $A\%$ максимально возможного, равного N^2.

Реализовать алгоритм при следующих параметрах: клеточное пространство 256×256 ячеек, $P_{\max} = 10$, $r = 1$, $A = 30\%$, $L = 15$, $T = 3$, $\Delta p = 5$, $p_1 = 35$, $\Delta e = 2$, $\Delta r = 3$. Выявить характерные зависимости в поведении колонии одноклеточных.

4.2 Ход работы

Начальное состояние системы показано на рисунке 19. В начальный момент времени запас питательных веществ максимален.

Красным цветом обозначены организмы с градиентом цвета от низкой к высокой энергии (чем ярче цвет, тем выше энергия). Зеленым цветом обозначены питательные вещества с градиентом цвета от низкого к высокому содержанию (чем ярче цвет, тем больше питательных веществ).

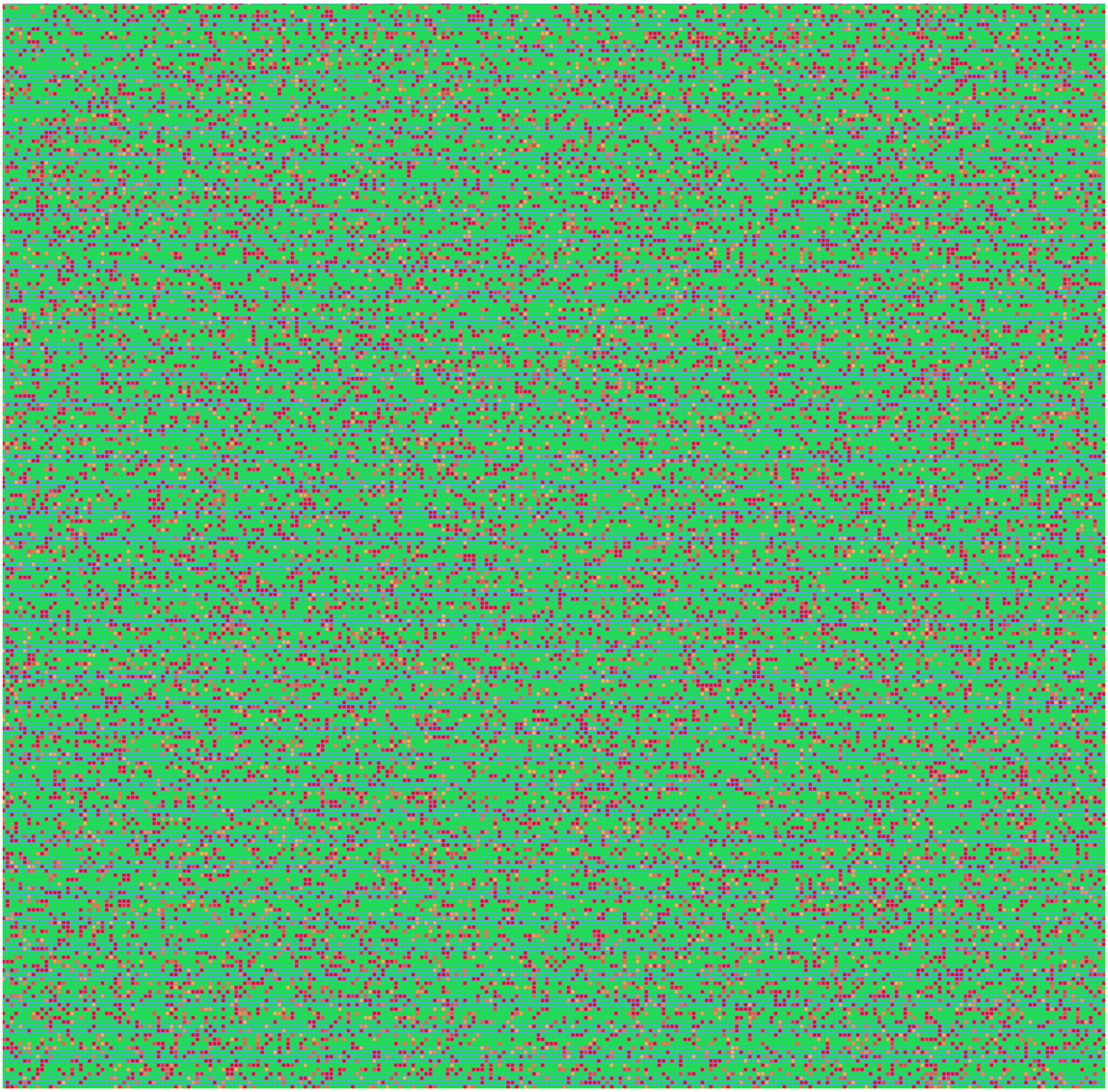

Рисунок 19 — Начальное состояние системы (28050 организмов)

Вследствие высокой начальной плотности популяции и активного размножения наступает стадия перенаселения (рисунок 20).

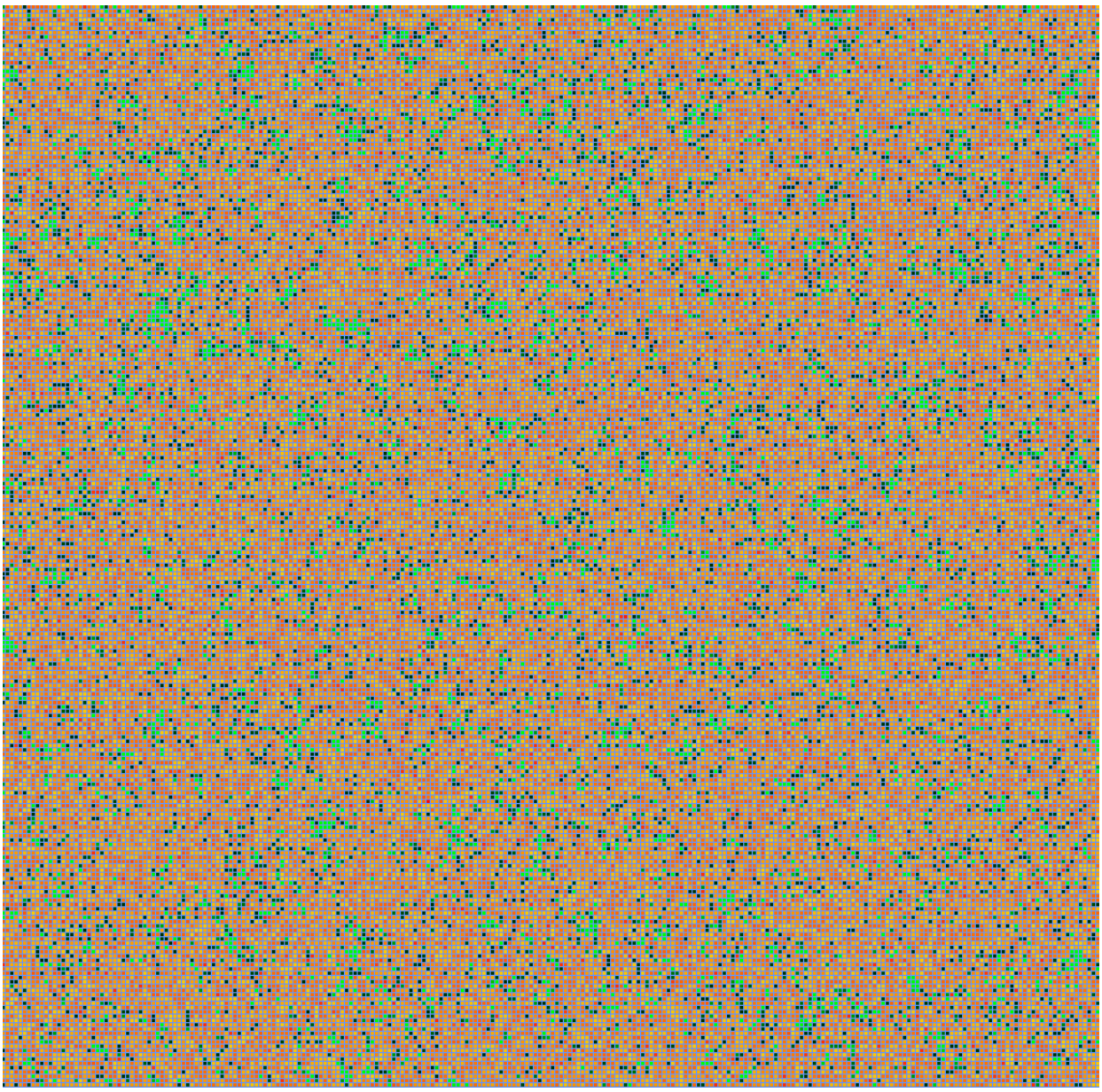

Рисунок 20 — Перенаселение на 5-м такте (61281 организмов)

Скорость потребления энергии популяцией значительно превышает скорость восстановления среды. Питательный ресурс истощается практически до нуля, что приводит к остановке деления и началу массового голода (рисунок 21).

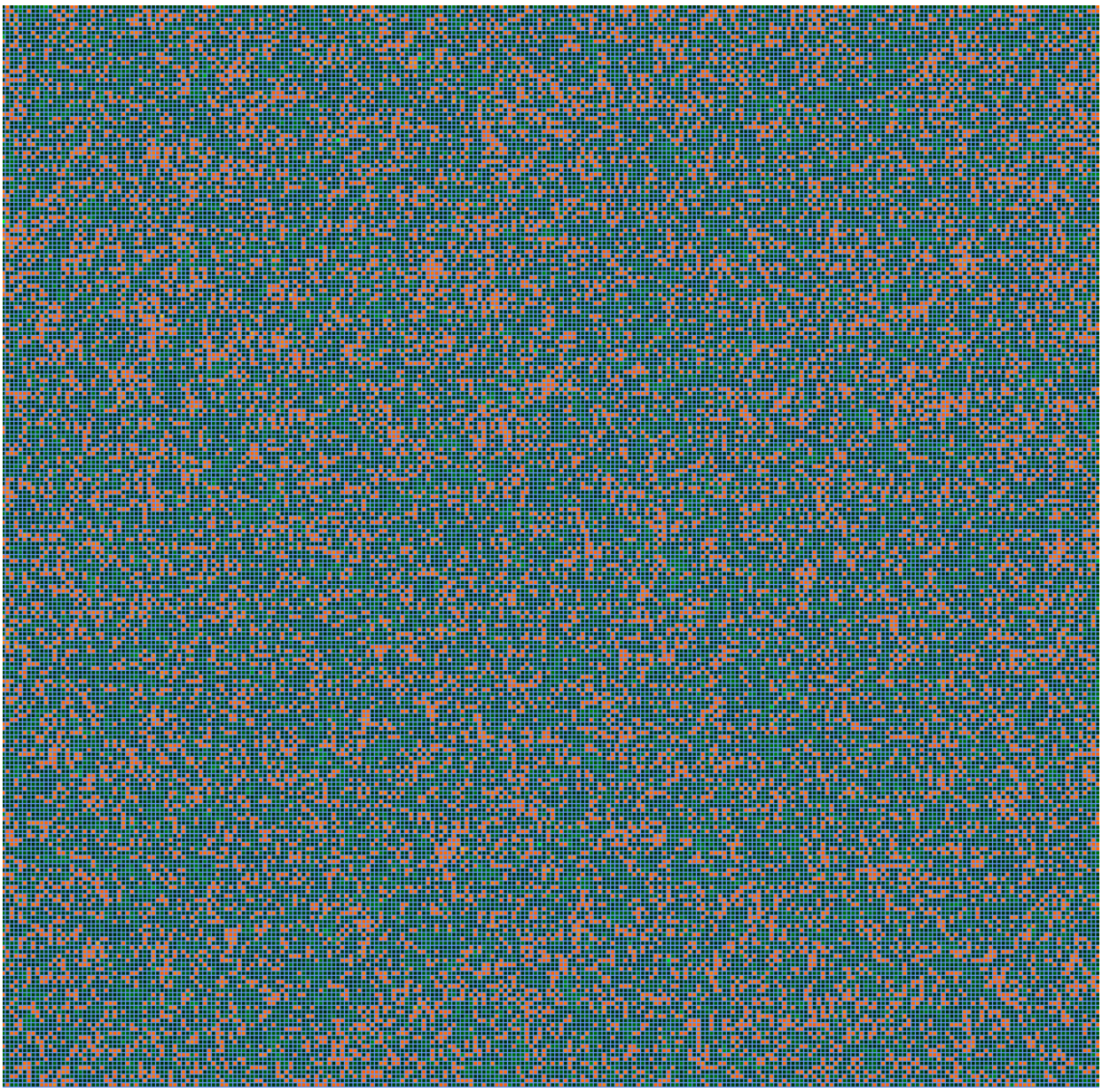

Рисунок 21 — Голод на 12-м такте (12858 организмов)

Далее система переходит в состояние динамического равновесия — периодическое повторение предыдущих стадий с примерным колебанием количества организмов от 25000 до 29000.

5 Модифицированный клеточный автомат «Организмы - питательная среда»

5.1 Постановка задачи

Выполнить модификацию алгоритма из задачи III, заменив правило 10 на следующее: особь всегда старается перейти на соседнюю свободную

клетку с наибольшим уровнем энергоемкости. Если ячейки в окрестности, имеют меньший запас энергии, то особь остается в прежней клетке.

5.2 Ход работы

При таком наборе правил система также проходит перенаселение и голод. На рисунке показано состояние модифицированной системы после 90 тактов.

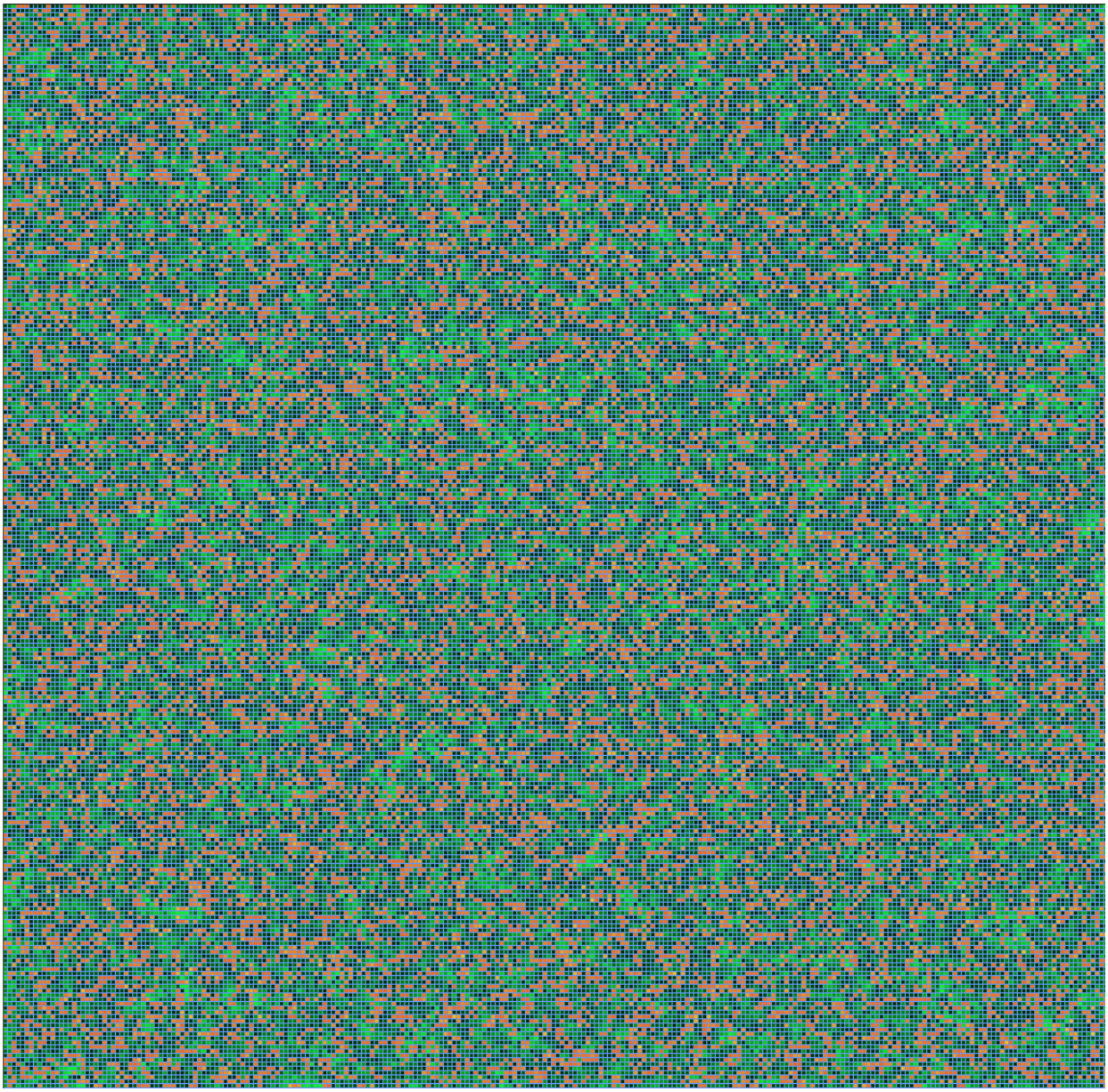

Рисунок 22 — Модифицированная система (18590 организмов)

Далее система также переходит в состояние динамического равновесия — периодическое повторение предыдущих стадий с колебанием количества организмов от 17000 до 19000.

6 Вывод

В ходе выполнения лабораторной работы были получены навыки имитационного моделирования сложных динамических систем с использованием клеточных автоматов «Жизнь», «Нейронная сеть» и «Организмы – питательная среда».

В ходе реализации клеточного автомата «Жизнь» были подобраны начальные распределения, при которых образуются стационарные и циклические структуры для разных правил.

В клеточном автомате «Нейронная сеть» наблюдаются круговые волны, которые создает периодический источник возбуждения и плоская волна. При столкновении волны «гасят» друг друга и продолжают своё как единое целое.

При реализации клеточного автомата «Организмы – питательная среда» были выявлены следующие особенности:

- Система демонстрирует автоколебательный режим с амплитудой колебаний численности до 24-25 тысяч особей.

- В модифицированной системе амплитуда аколебанияй существенно уменьшилась, что свидетельствует о более эффективном использовании ресурсов.

7 Листинг

main.cpp

```cpp
#include <iostream>
#include <memory>

#include "live.h"
#include "neural.h"
#include "organism.h"

using namespace std;

void task1() {  //"B3S23" or "B2S012"
        auto live = make_unique<GameLive>(
        GameRule::B2S012);  // default parameters from consts LiveGameConsts
        live->run();
}

void task2() {
        auto neural_network =
        make_unique<NeuralNetwork>();  // default parameters from consts LiveGameConsts
        neural_network->run();
}

void task3() {
        cout << "task 3" << endl;
        auto organism = make_unique<Organism>();  // default parameters from consts
            LiveGameConsts
        organism->run();
}

int main() {
        unsigned task;
        bool running = true;

        while (running) {
                cout << "Choose:" << endl;
                cout << "1 - Game of Life" << endl;
                cout << "2 - Neural Network" << endl;
                cout << "3 - Organism" << endl;
                cout << "0 - Exit" << endl;
                cout << "Write number ";
                cin >> task;

                switch (task) {
                        case 1:
                        task1();
                        break;

                        case 2:
                        task2();
                        break;

                        case 3:
                        task3();
                        break;

                        case 0:
                        running = false;
                        break;

                        default:
                        break;
                }
        }

        return 0;
}
```

base.h

```cpp
#pragma once

#include <SFML/Graphics.hpp>
#include <chrono>
#include <iostream>
#include <memory>
#include <vector>

#include "consts.h"
#include "grid.h"

using namespace std;

class BaseCellAutomaton {
        protected:
        unsigned num_cells;
        unsigned cell_size;
        bool is_running;
        float update_interval;
        chrono::steady_clock::time_point last_update;

        sf::Color background_color;
        unique_ptr<Grid> grid;
        vector<vector<sf::Color>> cell_colors;

        virtual void initGrid() = 0;
        virtual void updateGrid(bool force_update) = 0;
        virtual void updateCellColors() = 0;
        virtual void handleKeyPress(const sf::Event::KeyPressed& key_event);

        void drawGrid(sf::RenderWindow& window);
        void drawCells(sf::RenderWindow& window);

        public:
        BaseCellAutomaton(unsigned num_cells, unsigned cell_size,
        float default_update_interval = Consts::DEFAULT_UPDATE_INTERVAL);
        virtual ~BaseCellAutomaton();

        virtual void run();
        void increaseSpeed();
        void decreaseSpeed();
        void setRunning(bool running) { is_running = running; }
        bool isRunning() const { return is_running; }

        virtual void reset() = 0;
};
```

base.cpp

```cpp
#include "base.h"

BaseCellAutomaton::BaseCellAutomaton(unsigned num_cells, unsigned cell_size,
float default_update_interval)
: num_cells(num_cells),
cell_size(cell_size),
is_running(false),
update_interval(default_update_interval),
background_color(Consts::BACKGROUND_COLOR) {
        grid = make_unique<Grid>(num_cells, cell_size, Consts::GRID_COLOR);
        grid->init();
        cell_colors.resize(num_cells, vector<sf::Color>(num_cells,
            Consts::BACKGROUND_COLOR));

        cout << "BaseCellAutomaton init" << endl;
        last_update = chrono::steady_clock::now();
}

BaseCellAutomaton::~BaseCellAutomaton() { cout << "BaseCellAutomaton destroyed" << endl; }

void BaseCellAutomaton::drawGrid(sf::RenderWindow& window) {
        if (grid) {
```

```
                grid->drawGrid(window);
        }
}

void BaseCellAutomaton::drawCells(sf::RenderWindow& window) {
        if (grid) {
                grid->drawCells(window, cell_colors);
        }
}

void BaseCellAutomaton::increaseSpeed() {
        if (update_interval > Consts::MAX_SPEED) {
                update_interval -= Consts::SPEED_STEP;
                cout << "speed increased = " << update_interval << "s" << endl;
        }
}

void BaseCellAutomaton::decreaseSpeed() {
        if (update_interval < Consts::MIN_SPEED) {
                update_interval += Consts::SPEED_STEP;
                cout << "speed decreased = " << update_interval << "s" << endl;
        }
}

void BaseCellAutomaton::handleKeyPress(const sf::Event::KeyPressed& key_event) {
        switch (key_event.scancode) {
                case sf::Keyboard::Scan::Escape:
                break;
                case sf::Keyboard::Scan::Space:
                is_running = !is_running;
                break;
                case sf::Keyboard::Scan::Equal:  // increase speed
                increaseSpeed();
                break;
                case sf::Keyboard::Scan::Hyphen:  // decrease speed
                decreaseSpeed();
                break;
                case sf::Keyboard::Scan::Right:  // one next step
                updateGrid(true);
                updateCellColors();
                cout << "step forward" << endl;
                break;
                default:
                break;
        }
}

void BaseCellAutomaton::run() {
        sf::RenderWindow window(sf::VideoMode({grid->getWidth(), grid->getHeight()}),
            "Cell Automaton");
        window.setFramerateLimit(Consts::FRAME_LIMIT);

        while (window.isOpen()) {
                while (auto event = window.pollEvent()) {
                        if (event->is<sf::Event::Closed>()) {
                                window.close();
                        } else if (auto* key_event =
                            event->getIf<sf::Event::KeyPressed>()) {
                                if (key_event->scancode == sf::Keyboard::Scan::Escape) {
                                        window.close();
                                } else {
                                        handleKeyPress(*key_event);
                                }
                        }
                }

                auto now = chrono::steady_clock::now();
                chrono::duration<float> elapsed = now - last_update;

                if (is_running && elapsed.count() >= update_interval) {
                        updateGrid(false);
                        updateCellColors();
                        last_update = now;
```

```
                }

                window.clear(background_color);
                drawGrid(window);
                drawCells(window);
                window.display();
        }
}
```

live.h

```
#pragma once

#include <SFML/Graphics.hpp>
#include <chrono>
#include <iostream>
#include <memory>
#include <string>
#include <vector>

#include "base.h"
#include "consts.h"
#include "pattern.h"

using namespace std;

enum class GameRule { B3S23, B2S012 };

class GameLive : public BaseCellAutomaton {
        private:
        vector<vector<bool>> grid;
        vector<vector<bool>> next_grid;
        string current_pattern;
        unique_ptr<Pattern> pattern_manager;
        sf::Color cell_color;
        GameRule current_rule;

        sf::Color alive_color;
        sf::Color dead_color;

        void initGrid() override;
        void updateGrid(bool force_update) override;
        void updateCellColors() override;
        void handleKeyPress(const sf::Event::KeyPressed& key_event) override;

        int countNeighbors(unsigned x, unsigned y);
        void clearGrid();
        void toggleCell(unsigned x, unsigned y);
        void setCell(unsigned x, unsigned y, bool state = true);

        bool applyConwayRules(bool is_alive, int neighbors);
        bool applyB2S012Rules(bool is_alive, int neighbors);

        public:
        GameLive(GameRule rule = GameRule::B3S23,
        unsigned num_cells = LiveGameConsts::DEFAULT_NUM_CELLS,
        unsigned cell_size = LiveGameConsts::DEFAULT_CELL_SIZE,
        const string& pattern = "random");
        ~GameLive() override;

        void loadPattern(const string& pattern_name);
        void randomizeGrid();
        void reset() override;
        void setRule(GameRule rule);
        GameRule getRule() const { return current_rule; }

        vector<tuple<string, string, unsigned>> getAvailablePatterns() const;
};
```

live.cpp

```
#include "live.h"
```

```

GameLive::GameLive(GameRule rule, unsigned num_cells, unsigned cell_size, const string& pattern)
: BaseCellAutomaton(num_cells, cell_size, Consts::DEFAULT_UPDATE_INTERVAL),
current_pattern(pattern),
current_rule(rule),
alive_color(LiveGameConsts::CELL_COLOR),
dead_color(Consts::BACKGROUND_COLOR) {
        grid.resize(num_cells, vector<bool>(num_cells, false));
        next_grid.resize(num_cells, vector<bool>(num_cells, false));

        pattern_manager = make_unique<Pattern>(num_cells);

        initGrid();
        updateCellColors();

        cout << "GameLive init with rule: " << (rule == GameRule::B3S23 ? "B3S23" : "B2S012") << endl;
        cout << "Available patterns: " << getAvailablePatterns().size() << endl;
}

GameLive::~GameLive() { cout << "GameLive destroyed" << endl; }

void GameLive::initGrid() { pattern_manager->applyPattern(current_pattern, grid, current_rule); }

void GameLive::clearGrid() {
        for (unsigned y = 0; y < num_cells; y++) {
                for (unsigned x = 0; x < num_cells; x++) {
                        cell_colors[y][x] = dead_color;
                }
        }
        updateCellColors();
}

void GameLive::updateCellColors() {
        for (unsigned y = 0; y < num_cells; y++) {
                for (unsigned x = 0; x < num_cells; x++) {
                        if (grid[y][x]) {
                                cell_colors[y][x] = alive_color;
                        } else {
                                cell_colors[y][x] = dead_color;
                        }
                }
        }
}

int GameLive::countNeighbors(unsigned x, unsigned y) {
        int count = 0;

        for (int dy = -1; dy <= 1; dy++) {
                for (int dx = -1; dx <= 1; dx++) {
                        if (dx == 0 && dy == 0) continue;

                        int nx = static_cast<int>(x) + dx;
                        int ny = static_cast<int>(y) + dy;

                        if (nx >= 0 && nx < static_cast<int>(num_cells) && ny >= 0 &&
                        ny < static_cast<int>(num_cells)) {
                                if (grid[ny][nx]) {
                                        count++;
                                }
                        }
                }
        }

        return count;
}

bool GameLive::applyConwayRules(bool is_alive, int neighbors) {
        if (is_alive) {
                return (neighbors == 2 || neighbors == 3);
        } else {
```

```
                return (neighbors == 3);
        }
}

bool GameLive::applyB2S012Rules(bool is_alive, int neighbors) {
        if (is_alive) {
                return (neighbors == 0 || neighbors == 1 || neighbors == 2);
        } else {
                return (neighbors == 2);
        }
}

void GameLive::updateGrid(bool force_update) {
        if (!is_running && !force_update) {
                return;
        }

        for (unsigned y = 0; y < num_cells; y++) {
                for (unsigned x = 0; x < num_cells; x++) {
                        int neighbors = countNeighbors(x, y);

                        if (current_rule == GameRule::B3S23) {
                                next_grid[y][x] = applyConwayRules(grid[y][x], neighbors);
                        } else {
                                next_grid[y][x] = applyB2S012Rules(grid[y][x], neighbors);
                        }
                }
        }

        grid.swap(next_grid);
}

void GameLive::randomizeGrid() { pattern_manager->randomizeGrid(grid); }

void GameLive::setCell(unsigned x, unsigned y, bool state) {
        if (x < num_cells && y < num_cells) {
                grid[y][x] = state;
        }
}

void GameLive::toggleCell(unsigned x, unsigned y) {
        if (x < num_cells && y < num_cells) {
                grid[y][x] = !grid[y][x];
        }
}

void GameLive::loadPattern(const string& pattern_name) {
        current_pattern = pattern_name;
        pattern_manager->applyPattern(pattern_name, grid, current_rule);
}

void GameLive::reset() {
        clearGrid();
        is_running = false;
        updateCellColors();
}

void GameLive::setRule(GameRule rule) {
        if (current_rule != rule) {
                current_rule = rule;
                clearGrid();
                cout << "rule changed to " << (rule == GameRule::B3S23 ? "B3S23" :
                    "B2S012") << endl;
                updateCellColors();
        }
}

vector<tuple<string, string, unsigned>> GameLive::getAvailablePatterns() const {
        if (current_rule == GameRule::B3S23) {
                return LiveGameConsts::CONWAY_PATTERNS;
        } else {
                return LiveGameConsts::B2S012_PATTERNS;
        }
```

```
}

void GameLive::handleKeyPress(const sf::Event::KeyPressed& key_event) {
        static unsigned current_pattern_index = 0;
        auto available_patterns = getAvailablePatterns();

        switch (key_event.scancode) {
                case sf::Keyboard::Scan::R:
                randomizeGrid();
                is_running = false;
                current_pattern_index = 0;
                updateCellColors();
                cout << "grid randomized" << endl;
                break;

                case sf::Keyboard::Scan::C:
                clearGrid();
                is_running = false;
                updateCellColors();
                cout << "grid cleared" << endl;
                break;

                case sf::Keyboard::Scan::G:
                if (current_rule == GameRule::B3S23) {
                        setRule(GameRule::B2S012);
                } else {
                        setRule(GameRule::B3S23);
                }
                is_running = false;
                current_pattern_index = 0;
                loadPattern("random");
                updateCellColors();
                break;

                case sf::Keyboard::Scan::Num1:
                case sf::Keyboard::Scan::Num2:
                case sf::Keyboard::Scan::Num3:
                case sf::Keyboard::Scan::Num4:
                case sf::Keyboard::Scan::Num5:
                case sf::Keyboard::Scan::Num6:
                case sf::Keyboard::Scan::Num7:
                case sf::Keyboard::Scan::Num8:
                case sf::Keyboard::Scan::Num9: {
                        int index =
                        static_cast<int>(key_event.scancode) -
                            static_cast<int>(sf::Keyboard::Scan::Num1);
                        if (index >= 0 && index <
                            static_cast<int>(available_patterns.size())) {
                                current_pattern_index = index;
                                loadPattern(get<0>(available_patterns[current_pattern_index]));
                                is_running = false;
                                updateCellColors();
                                cout << "pattern loaded: " <<
                                    get<1>(available_patterns[current_pattern_index])
                                << endl;
                        }
                        break;
                }

                default:
                BaseCellAutomaton::handleKeyPress(key_event);
                break;
        }
}
```

pattern.h

```
#pragma once

#include <functional>
#include <iostream>
#include <random>
```

```
#include <string>
#include <vector>

using namespace std;

enum class GameRule;

class Pattern {
        private:
        unsigned num_cells;
        mt19937 rng;
        void setCell(vector<vector<bool>>& grid, unsigned x, unsigned y, bool state = true);
        void fillStripedBackground(vector<vector<bool>>& grid);

        public:
        Pattern(unsigned num_cells);
        void applyPattern(const string& pattern_name, vector<vector<bool>>& grid, GameRule rule);
        void randomizeGrid(vector<vector<bool>>& grid, int percentage = 20);
        void clearGrid(vector<vector<bool>>& grid);

        // B3S23
        void addBlock(vector<vector<bool>>& grid, unsigned center_x, unsigned center_y);
        void addBeehive(vector<vector<bool>>& grid, unsigned center_x, unsigned center_y);
        void addLoaf(vector<vector<bool>>& grid, unsigned center_x, unsigned center_y);
        void addBlinker(vector<vector<bool>>& grid, unsigned center_x, unsigned center_y);
        void addToad(vector<vector<bool>>& grid, unsigned center_x, unsigned center_y);
        void addGlider(vector<vector<bool>>& grid, unsigned center_x, unsigned center_y);
        void addPulsar(vector<vector<bool>>& grid, unsigned center_x, unsigned center_y);
        void addGosperGliderGun(vector<vector<bool>>& grid, unsigned center_x, unsigned center_y);

        // B2S012
        void addVerticalElement(vector<vector<bool>>& grid, unsigned center_x, unsigned center_y);
        void addEyePattern(vector<vector<bool>>& grid, unsigned center_x, unsigned center_y);
        void addEightPattern(vector<vector<bool>>& grid, unsigned center_x, unsigned center_y);
        void addNailPattern(vector<vector<bool>>& grid, unsigned center_x, unsigned center_y);
        void addGatesPattern(vector<vector<bool>>& grid, unsigned center_x, unsigned center_y);
        void addBackgroundWithVertical(vector<vector<bool>>& grid, unsigned center_x,
        unsigned center_y);
        void addBackgroundWithEye(vector<vector<bool>>& grid, unsigned center_x, unsigned center_y);
        void addBackgroundWithEight(vector<vector<bool>>& grid, unsigned center_x, unsigned center_y);
        void addBackgroundWithNail(vector<vector<bool>>& grid, unsigned center_x, unsigned center_y);
        void addBackgroundWithGates(vector<vector<bool>>& grid, unsigned center_x, unsigned center_y);
};
```

pattern.cpp

```
#include "pattern.h"

enum class GameRule { B3S23, B2S012 };

Pattern::Pattern(unsigned num_cells) : num_cells(num_cells) {
        random_device rd;
        rng = mt19937(rd());
}

void Pattern::setCell(vector<vector<bool>>& grid, unsigned x, unsigned y, bool state) {
        if (x < num_cells && y < num_cells) {
                grid[y][x] = state;
        }
}
```

```
void Pattern::randomizeGrid(vector<vector<bool>>& grid, int percentage) {
        uniform_int_distribution<> dis(0, 99);

        for (unsigned y = 0; y < num_cells; y++) {
                for (unsigned x = 0; x < num_cells; x++) {
                        grid[y][x] = (dis(rng) < percentage);
                }
        }
}

void Pattern::clearGrid(vector<vector<bool>>& grid) {
        for (unsigned y = 0; y < num_cells; y++) {
                for (unsigned x = 0; x < num_cells; x++) {
                        grid[y][x] = false;
                }
        }
}

void Pattern::fillStripedBackground(vector<vector<bool>>& grid) {
        for (unsigned r = 0; r < num_cells; r += 2) {
                for (unsigned c = 0; c < num_cells; c++) {
                        setCell(grid, c, r, true);
                }
        }

        if (num_cells % 2 == 0) {
                for (unsigned c = 0; c < num_cells; c++) {
                        setCell(grid, c, num_cells - 2, false);
                }
                unsigned last_filled_row = num_cells - 2;
                setCell(grid, 1, num_cells - 2);
                setCell(grid, num_cells - 1, num_cells - 2);
        }
}

void Pattern::applyPattern(const string& pattern_name, vector<vector<bool>>& grid,
    GameRule rule) {
        clearGrid(grid);

        if (pattern_name == "random" || pattern_name == "rand") {
                randomizeGrid(grid);
                return;
        }

        unsigned center_x = num_cells / 2;
        unsigned center_y = num_cells / 2;

        if (rule != GameRule::B3S23) {
                if (pattern_name == "striped_background") {
                        fillStripedBackground(grid);
                        return;
                } else if (pattern_name == "vertical_element") {
                        addVerticalElement(grid, center_x, center_y);
                        return;
                } else if (pattern_name == "background_with_vertical") {
                        addBackgroundWithVertical(grid, center_x, center_y);
                        return;
                } else if (pattern_name == "background_with_eye") {
                        addBackgroundWithEye(grid, center_x, center_y);
                        return;
                } else if (pattern_name == "background_with_eight") {
                        addBackgroundWithEight(grid, center_x, center_y);
                        return;
                } else if (pattern_name == "background_with_nail") {
                        addBackgroundWithNail(grid, center_x, center_y);
                        return;
                } else if (pattern_name == "background_with_gates") {
                        addBackgroundWithGates(grid, center_x, center_y);
                        return;
                }
        }

```

```
        if (pattern_name == "block") {
                addBlock(grid, center_x, center_y);
        } else if (pattern_name == "beehive") {
                addBeehive(grid, center_x, center_y);
        } else if (pattern_name == "loaf") {
                addLoaf(grid, center_x, center_y);
        } else if (pattern_name == "blinker") {
                addBlinker(grid, center_x, center_y);
        } else if (pattern_name == "toad") {
                addToad(grid, center_x, center_y);
        } else if (pattern_name == "glider") {
                addGlider(grid, center_x, center_y);
        } else if (pattern_name == "pulsar") {
                addPulsar(grid, center_x, center_y);
        } else if (pattern_name == "gosper_glider_gun") {
                addGosperGliderGun(grid, center_x, center_y);
        } else {
                randomizeGrid(grid);
        }
}

void Pattern::addBlock(vector<vector<bool>>& grid, unsigned cx, unsigned cy) {
        setCell(grid, cx, cy);
        setCell(grid, cx + 1, cy);
        setCell(grid, cx, cy + 1);
        setCell(grid, cx + 1, cy + 1);
}

void Pattern::addBeehive(vector<vector<bool>>& grid, unsigned cx, unsigned cy) {
        setCell(grid, cx, cy);
        setCell(grid, cx + 1, cy - 1);
        setCell(grid, cx + 1, cy + 1);
        setCell(grid, cx + 2, cy - 1);
        setCell(grid, cx + 2, cy + 1);
        setCell(grid, cx + 3, cy);
}

void Pattern::addLoaf(vector<vector<bool>>& grid, unsigned cx, unsigned cy) {
        setCell(grid, cx, cy);
        setCell(grid, cx + 1, cy - 1);
        setCell(grid, cx + 1, cy + 1);
        setCell(grid, cx + 2, cy - 2);
        setCell(grid, cx + 2, cy + 1);
        setCell(grid, cx + 3, cy - 1);
        setCell(grid, cx + 3, cy);
}

void Pattern::addBlinker(vector<vector<bool>>& grid, unsigned cx, unsigned cy) {
        setCell(grid, cx, cy - 1);
        setCell(grid, cx, cy);
        setCell(grid, cx, cy + 1);
}

void Pattern::addToad(vector<vector<bool>>& grid, unsigned cx, unsigned cy) {
        setCell(grid, cx - 1, cy);
        setCell(grid, cx, cy - 1);
        setCell(grid, cx, cy);
        setCell(grid, cx, cy + 1);
        setCell(grid, cx + 1, cy - 1);
        setCell(grid, cx + 1, cy);
        setCell(grid, cx + 1, cy + 1);
        setCell(grid, cx + 2, cy);
}

void Pattern::addGlider(vector<vector<bool>>& grid, unsigned cx, unsigned cy) {
        setCell(grid, cx + 1, cy);
        setCell(grid, cx + 2, cy + 1);
        setCell(grid, cx, cy + 2);
        setCell(grid, cx + 1, cy + 2);
        setCell(grid, cx + 2, cy + 2);
}

void Pattern::addPulsar(vector<vector<bool>>& grid, unsigned cx, unsigned cy) {
```

```
        int px = cx - 7;
        int py = cy - 7;

        for (int i = 0; i < 3; i++) {
                setCell(grid, px + 2, py + 4 + i);
                setCell(grid, px + 7, py + 4 + i);
                setCell(grid, px + 9, py + 4 + i);
                setCell(grid, px + 14, py + 4 + i);
        }

        for (int i = 0; i < 3; i++) {
                setCell(grid, px + 4 + i, py + 2);
                setCell(grid, px + 4 + i, py + 7);
                setCell(grid, px + 4 + i, py + 9);
                setCell(grid, px + 4 + i, py + 14);
        }
}

void Pattern::addGosperGliderGun(vector<vector<bool>>& grid, unsigned cx, unsigned cy) {
        int gx = cx - 18;
        int gy = cy - 9;

        setCell(grid, gx + 24, gy + 0);
        setCell(grid, gx + 22, gy + 1);
        setCell(grid, gx + 24, gy + 1);
        setCell(grid, gx + 12, gy + 2);
        setCell(grid, gx + 13, gy + 2);
        setCell(grid, gx + 20, gy + 2);
        setCell(grid, gx + 21, gy + 2);
        setCell(grid, gx + 34, gy + 2);
        setCell(grid, gx + 35, gy + 2);
        setCell(grid, gx + 11, gy + 3);
        setCell(grid, gx + 15, gy + 3);
        setCell(grid, gx + 20, gy + 3);
        setCell(grid, gx + 21, gy + 3);
        setCell(grid, gx + 34, gy + 3);
        setCell(grid, gx + 35, gy + 3);
        setCell(grid, gx + 0, gy + 4);
        setCell(grid, gx + 1, gy + 4);
        setCell(grid, gx + 10, gy + 4);
        setCell(grid, gx + 16, gy + 4);
        setCell(grid, gx + 20, gy + 4);
        setCell(grid, gx + 21, gy + 4);
        setCell(grid, gx + 0, gy + 5);
        setCell(grid, gx + 1, gy + 5);
        setCell(grid, gx + 10, gy + 5);
        setCell(grid, gx + 14, gy + 5);
        setCell(grid, gx + 16, gy + 5);
        setCell(grid, gx + 17, gy + 5);
        setCell(grid, gx + 22, gy + 5);
        setCell(grid, gx + 24, gy + 5);
        setCell(grid, gx + 10, gy + 6);
        setCell(grid, gx + 16, gy + 6);
        setCell(grid, gx + 24, gy + 6);
        setCell(grid, gx + 11, gy + 7);
        setCell(grid, gx + 15, gy + 7);
        setCell(grid, gx + 12, gy + 8);
        setCell(grid, gx + 13, gy + 8);
}

void Pattern::addVerticalElement(vector<vector<bool>>& grid, unsigned cx, unsigned cy) {
        int offset_x = static_cast<int>(cx) - 12;
        int offset_y = static_cast<int>(cy) - 12;

        setCell(grid, offset_x + 10, offset_y + 10, false);
        setCell(grid, offset_x + 10, offset_y + 12, false);
        setCell(grid, offset_x + 11, offset_y + 11, true);
        setCell(grid, offset_x + 12, offset_y + 10, false);
        setCell(grid, offset_x + 12, offset_y + 12, false);
}

void Pattern::addEyePattern(vector<vector<bool>>& grid, unsigned cx, unsigned cy) {
        int offset_x = static_cast<int>(cx) - 12;
```

```
        int offset_y = static_cast<int>(cy) - 12;

        setCell(grid, offset_x + 5, offset_y + 4, false);
        setCell(grid, offset_x + 6, offset_y + 4, false);
        setCell(grid, offset_x + 5, offset_y + 6, false);
        setCell(grid, offset_x + 6, offset_y + 6, false);
}

void Pattern::addEightPattern(vector<vector<bool>>& grid, unsigned cx, unsigned cy) {
        int offset_x = static_cast<int>(cx) - 12;
        int offset_y = static_cast<int>(cy) - 12;

        setCell(grid, offset_x + 20, offset_y + 4, false);
        setCell(grid, offset_x + 21, offset_y + 4, false);
        setCell(grid, offset_x + 20, offset_y + 6, false);
        setCell(grid, offset_x + 21, offset_y + 6, false);
        setCell(grid, offset_x + 20, offset_y + 8, false);
        setCell(grid, offset_x + 21, offset_y + 8, false);
        setCell(grid, offset_x + 20, offset_y + 10, false);
        setCell(grid, offset_x + 21, offset_y + 10, false);
}

void Pattern::addNailPattern(vector<vector<bool>>& grid, unsigned cx, unsigned cy) {
        int offset_x = static_cast<int>(cx) - 12;
        int offset_y = static_cast<int>(cy) - 12;

        setCell(grid, offset_x + 5, offset_y + 14, false);
        setCell(grid, offset_x + 6, offset_y + 14, false);
        setCell(grid, offset_x + 7, offset_y + 14, false);
        setCell(grid, offset_x + 6, offset_y + 15, true);
}

void Pattern::addGatesPattern(vector<vector<bool>>& grid, unsigned cx, unsigned cy) {
        int offset_x = static_cast<int>(cx) - 12;
        int offset_y = static_cast<int>(cy) - 12;

        setCell(grid, offset_x + 20, offset_y + 15, true);
        setCell(grid, offset_x + 21, offset_y + 14, false);
        setCell(grid, offset_x + 23, offset_y + 14, false);
        setCell(grid, offset_x + 21, offset_y + 16, false);
        setCell(grid, offset_x + 23, offset_y + 16, false);
}

void Pattern::addBackgroundWithVertical(vector<vector<bool>>& grid, unsigned cx, unsigned
    cy) {
        fillStripedBackground(grid);
        addVerticalElement(grid, cx, cy);
}

void Pattern::addBackgroundWithEye(vector<vector<bool>>& grid, unsigned cx, unsigned cy) {
        fillStripedBackground(grid);
        addEyePattern(grid, cx, cy);
}

void Pattern::addBackgroundWithEight(vector<vector<bool>>& grid, unsigned cx, unsigned cy)
    {
        fillStripedBackground(grid);
        addEightPattern(grid, cx, cy);
}

void Pattern::addBackgroundWithNail(vector<vector<bool>>& grid, unsigned cx, unsigned cy) {
        fillStripedBackground(grid);
        addNailPattern(grid, cx, cy);
}

void Pattern::addBackgroundWithGates(vector<vector<bool>>& grid, unsigned cx, unsigned cy)
    {
        fillStripedBackground(grid);
        addGatesPattern(grid, cx, cy);
}
```

neural.h

```
#pragma once

#include <SFML/Graphics.hpp>
#include <chrono>
#include <iostream>
#include <memory>
#include <string>
#include <vector>

#include "base.h"
#include "consts.h"

using namespace std;

enum class CellState { REST, ACTIVE, RECOVERY };

class NeuralNetwork : public BaseCellAutomaton {
        private:
        vector<vector<CellState>> grid;
        vector<vector<CellState>> next_grid;
        vector<vector<float>> activator_level;
        vector<vector<int>> state_timer;
        unsigned int current_tick;

        sf::Color cell_color_active;
        sf::Color cell_color_rest;
        sf::Color cell_color_recovery;

        void initGrid() override;
        void updateCellColors() override;
        void updateGrid(bool force_update) override;
        void handleKeyPress(const sf::Event::KeyPressed& key_event) override;

        void initFlatFront();
        void initGenerator();

        public:
        NeuralNetwork(unsigned num_cells = NeuralNetworkConsts::DEFAULT_NUM_CELLS,
        unsigned cell_size = NeuralNetworkConsts::DEFAULT_CELL_SIZE);
        ~NeuralNetwork() override;
        void reset() override;
};
```

neural.cpp

```
#include "neural.h"

NeuralNetwork::NeuralNetwork(unsigned num_cells, unsigned cell_size)
: BaseCellAutomaton(num_cells, cell_size, Consts::DEFAULT_UPDATE_INTERVAL),
current_tick(0),
cell_color_active(NeuralNetworkConsts::CELL_COLOR_ACTIVE),
cell_color_rest(NeuralNetworkConsts::CELL_COLOR_REST),
cell_color_recovery(NeuralNetworkConsts::CELL_COLOR_RECOVERY) {
        grid.resize(num_cells, vector<CellState>(num_cells, CellState::REST));
        next_grid.resize(num_cells, vector<CellState>(num_cells, CellState::REST));
        activator_level.resize(num_cells, vector<float>(num_cells, 0.0f));
        state_timer.resize(num_cells, vector<int>(num_cells, 0));

        initGrid();
        updateCellColors();

        cout << "NeuralNetwork␣init" << endl;
}

NeuralNetwork::~NeuralNetwork() { cout << "NeuralNetwork␣destroyed" << endl; }

void NeuralNetwork::initGrid() { initFlatFront(); }

void NeuralNetwork::updateCellColors() {
        for (unsigned y = 0; y < num_cells; y++) {
                for (unsigned x = 0; x < num_cells; x++) {
                        switch (grid[y][x]) {
```

```
                                case CellState::ACTIVE:
                                cell_colors[y][x] = cell_color_active;
                                break;
                                case CellState::RECOVERY:
                                cell_colors[y][x] = cell_color_recovery;
                                break;
                                case CellState::REST:
                                cell_colors[y][x] = cell_color_rest;
                                break;
                        }
                }
        }
}

void NeuralNetwork::initFlatFront() {
        unsigned front_x = num_cells - num_cells / 5;

        for (unsigned y = 0; y < num_cells; y++) {
                for (unsigned dx = 0; dx < 3; dx++) {
                        unsigned x = front_x + dx;
                        if (x < num_cells) {
                                grid[y][x] = CellState::ACTIVE;
                                activator_level[y][x] = 1.0f;
                                state_timer[y][x] = NeuralNetworkConsts::T;
                        }
                }
        }
}

void NeuralNetwork::initGenerator() {
        int center_y = num_cells / 2;
        int center_x = num_cells / 2 - 60;

        for (int dy = -1; dy <= 1; dy++) {
                for (int dx = -1; dx <= 1; dx++) {
                        int y = center_y + dy;
                        int x = center_x + dx;

                        if (y >= 0 && y < static_cast<int>(num_cells) && x >= 0 &&
                        x < static_cast<int>(num_cells)) {
                                if (grid[y][x] == CellState::REST) {
                                        grid[y][x] = CellState::ACTIVE;
                                        activator_level[y][x] = 1.0f;
                                        state_timer[y][x] = NeuralNetworkConsts::T;
                                }
                        }
                }
        }
}

void NeuralNetwork::updateGrid(bool force_update) {
        if (!is_running && !force_update) {
                return;
        }

        current_tick++;

        vector<vector<CellState>> new_grid =
        vector<vector<CellState>>(num_cells, vector<CellState>(num_cells,
            CellState::REST));
        vector<vector<float>> new_activator =
        vector<vector<float>>(num_cells, vector<float>(num_cells, 0.0f));
        vector<vector<int>> new_timer = vector<vector<int>>(num_cells,
            vector<int>(num_cells, 0));

        const float decay = 1.0f - NeuralNetworkConsts::A;
        const int max_idx = static_cast<int>(num_cells);

        for (int y = 0; y < max_idx; ++y) {
                for (int x = 0; x < max_idx; ++x) {
                        int y1 = max(0, y - 1);
                        int y2 = min(max_idx, y + 2);
                        int x1 = max(0, x - 1);
```

```
                        int x2 = min(max_idx, x + 2);

                        float sum = 0.0f;
                        for (int ny = y1; ny < y2; ++ny) {
                                for (int nx = x1; nx < x2; ++nx) {
                                        sum += activator_level[ny][nx];
                                }
                        }

                        CellState state = grid[y][x];
                        int timer = state_timer[y][x];
                        float activator = activator_level[y][x] * decay;

                        switch (state) {
                                case CellState::REST:
                                if (sum >= NeuralNetworkConsts::P) {
                                        new_grid[y][x] = CellState::ACTIVE;
                                        new_timer[y][x] = NeuralNetworkConsts::T;
                                        new_activator[y][x] = 1.0f;
                                } else {
                                        new_grid[y][x] = CellState::REST;
                                        new_activator[y][x] = activator;
                                }
                                break;

                                case CellState::ACTIVE:
                                new_activator[y][x] = 1.0f;
                                if (timer > 1) {
                                        new_grid[y][x] = CellState::ACTIVE;
                                        new_timer[y][x] = timer - 1;
                                } else {
                                        new_grid[y][x] = CellState::RECOVERY;
                                        new_timer[y][x] = NeuralNetworkConsts::B;
                                }
                                break;

                                case CellState::RECOVERY:
                                new_activator[y][x] = activator;
                                if (timer > 1) {
                                        new_grid[y][x] = CellState::RECOVERY;
                                        new_timer[y][x] = timer - 1;
                                } else {
                                        new_grid[y][x] = CellState::REST;
                                }
                                break;
                        }
                }
        }

        grid = move(new_grid);
        activator_level = move(new_activator);
        state_timer = move(new_timer);

        if (current_tick == 1 || current_tick % NeuralNetworkConsts::SOURCE_PERIOD == 0) {
                initGenerator();
        }

        next_grid = grid;
}

void NeuralNetwork::handleKeyPress(const sf::Event::KeyPressed& key_event) {
        BaseCellAutomaton::handleKeyPress(key_event);

        switch (key_event.scancode) {
                case sf::Keyboard::Scan::R:
                reset();
                break;
                default:
                break;
        }
}

void NeuralNetwork::reset() {
```

```
        for (unsigned y = 0; y < num_cells; y++) {
                for (unsigned x = 0; x < num_cells; x++) {
                        grid[y][x] = CellState::REST;
                        activator_level[y][x] = 0.0f;
                        state_timer[y][x] = 0;
                }
        }
        current_tick = 0;
        is_running = false;
        initGrid();
        updateCellColors();
}
```

organism.h

```
#pragma once

#include <SFML/Graphics.hpp>
#include <chrono>
#include <iostream>
#include <memory>
#include <random>
#include <string>
#include <vector>

#include "base.h"
#include "consts.h"

using namespace std;

struct OrganismData {
        int age;
        float energy;
        bool is_alive;
        int x, y;

        OrganismData() : age(0), energy(0), is_alive(false), x(0), y(0) {}
        OrganismData(float e, int a, bool alive, int pos_x, int pos_y)
        : energy(e), age(a), is_alive(alive), x(pos_x), y(pos_y) {}
};

class Organism : public BaseCellAutomaton {
        private:
        vector<vector<bool>> grid;
        vector<vector<bool>> next_grid;
        vector<vector<float>> nutrients;
        vector<vector<float>> next_nutrients;
        vector<vector<OrganismData>> organisms;
        vector<vector<OrganismData>> next_organisms;

        sf::Color organism_color_high;
        sf::Color organism_color_medium;
        sf::Color organism_color_low;

        unsigned int tick;
        bool vision;

        mt19937 rng;
        uniform_int_distribution<unsigned> pos_dist;

        vector<pair<int, int>> getEmptyNeighbors(int x, int y, const vector<vector<bool>>&
            grid);
        pair<int, int> chooseBestMove(const vector<pair<int, int>>& neighbors, int
            current_x,
        int current_y, bool smart);
        void updateNutrients();
        void updateOrganisms();
        void updateSingleOrganism(int x, int y);
        unsigned int getPopulationCount() const;

        void initGrid() override;
        void updateGrid(bool force_update) override;
```

```
        void updateCellColors() override;
        void handleKeyPress(const sf::Event::KeyPressed& key_event) override;

        public:
        Organism(unsigned num_cells = OrganismsConsts::DEFAULT_NUM_CELLS,
        unsigned cell_size = OrganismsConsts::DEFAULT_CELL_SIZE);
        virtual ~Organism();

        void reset() override;
        void toggleVision();
};
```

organism.cpp

```
#include "organism.h"

Organism::Organism(unsigned num_cells, unsigned cell_size)
: BaseCellAutomaton(num_cells, cell_size, Consts::DEFAULT_UPDATE_INTERVAL),
organism_color_high(OrganismsConsts::ORGANISM_COLOR_HIGH),
organism_color_medium(OrganismsConsts::ORGANISM_COLOR_MEDIUM),
organism_color_low(OrganismsConsts::ORGANISM_COLOR_LOW),
tick(0),
vision(false),
rng(random_device{}()),
pos_dist(0, num_cells - 1) {
        grid.resize(num_cells, vector<bool>(num_cells, false));
        next_grid.resize(num_cells, vector<bool>(num_cells, false));
        nutrients.resize(num_cells, vector<float>(num_cells, OrganismsConsts::P_MAX));
        next_nutrients.resize(num_cells, vector<float>(num_cells, OrganismsConsts::P_MAX));
        organisms.resize(num_cells, vector<OrganismData>(num_cells));
        next_organisms.resize(num_cells, vector<OrganismData>(num_cells));

        initGrid();
        updateCellColors();

        cout << "Organisms␣init" << endl;
}

Organism::~Organism() { cout << "Organism␣destroyed" << endl; }

void Organism::initGrid() {
        for (unsigned y = 0; y < num_cells; y++) {
                for (unsigned x = 0; x < num_cells; x++) {
                        grid[y][x] = false;
                        nutrients[y][x] = OrganismsConsts::P_MAX;
                        organisms[y][x] = OrganismData(0, 0, false, x, y);
                }
        }

        unsigned total_cells = num_cells * num_cells;
        unsigned max_organisms = (total_cells * OrganismsConsts::A) / 100;
        unsigned placed = 0;

        // cout << "Initializing " << max_organisms << " organisms out of " << total_cells << " cells"
        //      << endl;

        while (placed < max_organisms) {
                int x = pos_dist(rng);
                int y = pos_dist(rng);

                if (!grid[y][x]) {
                        grid[y][x] = true;
                        int initial_age = uniform_int_distribution<int>(0, OrganismsConsts::T)(rng);
                        float initial_energy = uniform_real_distribution<float>(OrganismsConsts::DELTA_E * 5,
                        OrganismsConsts::P1)(rng);

                        organisms[y][x] = OrganismData(initial_energy, initial_age, true, x, y);
                        placed++;
```

```
                }
        }

        tick = 0;
}

void Organism::updateGrid(bool force_update) {
        if (!is_running && !force_update) {
                return;
        }

        updateNutrients();
        updateOrganisms();

        grid.swap(next_grid);
        nutrients.swap(next_nutrients);
        organisms.swap(next_organisms);

        tick++;

        if (tick % 1 == 0) {
                cout << "tick: " << tick << ", population: " << getPopulationCount() << endl;
        }
}

void Organism::updateNutrients() {
        // rule 4
        for (unsigned y = 0; y < num_cells; y++) {
                for (unsigned x = 0; x < num_cells; x++) {
                        if (nutrients[y][x] < OrganismsConsts::P_MAX) {
                                nutrients[y][x] = min(OrganismsConsts::P_MAX, nutrients[y][x] + OrganismsConsts::R);
                        }
                }
        }
}

void Organism::updateOrganisms() {
        for (unsigned y = 0; y < num_cells; y++) {
                for (unsigned x = 0; x < num_cells; x++) {
                        next_grid[y][x] = false;
                        next_nutrients[y][x] = nutrients[y][x];
                        next_organisms[y][x] = OrganismData(0, 0, false, x, y);
                }
        }

        vector<pair<int, int>> organisms_positions;
        for (unsigned y = 0; y < num_cells; y++) {
                for (unsigned x = 0; x < num_cells; x++) {
                        if (grid[y][x] && organisms[y][x].is_alive) {
                                organisms_positions.push_back({x, y});
                        }
                }
        }

        vector<bool> processed(organisms_positions.size(), false);
        int remaining = organisms_positions.size();

        while (remaining > 0) {
                int idx = uniform_int_distribution<int>(0, organisms_positions.size() - 1)(rng);
                if (!processed[idx]) {
                        processed[idx] = true;
                        remaining--;
                        updateSingleOrganism(organisms_positions[idx].first, organisms_positions[idx].second);
                }
        }
}

void Organism::updateSingleOrganism(int x, int y) {
        if (!grid[y][x] || !organisms[y][x].is_alive) return;
```

```
124
125         OrganismData org = organisms[y][x];
126
127         float current_nutrient = nutrients[y][x];
128
129         // rule 7 - 8
130         float can_take = min(OrganismsConsts::DELTA_P, OrganismsConsts::P1 - org.energy);
131         float actual_take = min(current_nutrient, can_take);
132
133         if (actual_take > 0) {
134                 next_nutrients[y][x] = max(0.0f, next_nutrients[y][x] - actual_take);
135                 org.energy += actual_take;
136         }
137
138         if (org.energy > OrganismsConsts::P1) {
139                 org.energy = OrganismsConsts::P1;
140         }
141
142         // rule 9
143         org.energy -= OrganismsConsts::DELTA_E;
144
145         // rule 11
146         org.age++;
147
148         // rule 12
149         if (org.age > OrganismsConsts::L || org.energy <= 0) {
150                 return;
151         }
152
153         vector<pair<int, int>> empty_neighbors = getEmptyNeighbors(x, y, next_grid);
154
155         // rule 13
156         if (org.age >= OrganismsConsts::T && org.energy > OrganismsConsts::DELTA_R &&
157         !empty_neighbors.empty()) {
158                 pair<int, int> target = chooseBestMove(empty_neighbors, x, y, vision);
159
160                 if (target.first != -1) {
161                         org.energy -= OrganismsConsts::DELTA_R;
162
163                         if (org.energy < 0) org.energy = 0;
164
165                         float child_energy = org.energy / 2.0f;
166                         org.energy -= child_energy;
167
168                         OrganismData child(max(0.0f, child_energy), 0, true, x, y);
169
170                         org.x = target.first;
171                         org.y = target.second;
172
173                         next_grid[target.second][target.first] = true;
174                         next_organisms[target.second][target.first] = org;
175                         next_grid[y][x] = true;
176                         next_organisms[y][x] = child;
177
178                         return;
179                 }
180         }
181
182         // rule 10
183         if (!empty_neighbors.empty()) {
184                 pair<int, int> target = chooseBestMove(empty_neighbors, x, y, vision);
185
186                 if (target.first != -1) {
187                         org.x = target.first;
188                         org.y = target.second;
189                         next_grid[target.second][target.first] = true;
190                         next_organisms[target.second][target.first] = org;
191                         return;
192                 }
193         }
194
195         if (!next_grid[y][x]) {
196                 next_grid[y][x] = true;
```

```
                next_organisms[y][x] = org;
        }
}

vector<pair<int, int>> Organism::getEmptyNeighbors(int x, int y, const vector<vector<bool>>& grid) {
        vector<pair<int, int>> empty;
        empty.reserve(8);

        for (int dy = -1; dy <= 1; dy++) {
                for (int dx = -1; dx <= 1; dx++) {
                        if (dx == 0 && dy == 0) continue;

                        int nx = x + dx;
                        int ny = y + dy;

                        if (nx >= 0 && nx < static_cast<int>(num_cells) && ny >= 0 &&
                        ny < static_cast<int>(num_cells) && !grid[ny][nx]) {
                                empty.push_back({nx, ny});
                        }
                }
        }

        return empty;
}

pair<int, int> Organism::chooseBestMove(const vector<pair<int, int>>& neighbors, int current_x,
int current_y, bool smart) {
        if (neighbors.empty()) {
                return {-1, -1};
        }

        if (!smart) {
                // task
                int idx = uniform_int_distribution<int>(0, neighbors.size() - 1)(rng);
                return neighbors[idx];
        }

        // task 4
        float max_nutrient = -1;
        vector<pair<int, int>> best_cells;

        for (const auto& pos : neighbors) {
                float nutrient_level = nutrients[pos.second][pos.first];
                if (nutrient_level > max_nutrient) {
                        max_nutrient = nutrient_level;
                        best_cells.clear();
                        best_cells.push_back(pos);
                } else if (abs(nutrient_level - max_nutrient) < 1e-6) {
                        best_cells.push_back(pos);
                }
        }

        float current_nutrient = nutrients[current_y][current_x];
        if (max_nutrient <= current_nutrient) {
                return {-1, -1};
        }

        int idx = uniform_int_distribution<int>(0, best_cells.size() - 1)(rng);
        return best_cells[idx];
}

void Organism::updateCellColors() {
        for (unsigned y = 0; y < num_cells; y++) {
                for (unsigned x = 0; x < num_cells; x++) {
                        if (grid[y][x] && organisms[y][x].is_alive) {
                                float energy_ratio = organisms[y][x].energy / OrganismsConsts::P1;

                                sf::Color org_color;
                                if (energy_ratio > 0.7f) {
                                        org_color = organism_color_high;
```

```
                                } else if (energy_ratio > 0.3f) {
                                        float t = (energy_ratio - 0.3f) / 0.4f;
                                        uint8_t r = organism_color_medium.r +
                                        t * (organism_color_high.r -
                                            organism_color_medium.r);
                                        uint8_t g = organism_color_medium.g +
                                        t * (organism_color_high.g -
                                            organism_color_medium.g);
                                        uint8_t b = organism_color_medium.b +
                                        t * (organism_color_high.b -
                                            organism_color_medium.b);
                                        cell_colors[y][x] = sf::Color(r, g, b);
                                } else {
                                        float t = energy_ratio / 0.3f;
                                        uint8_t r =
                                        organism_color_low.r + t *
                                            (organism_color_medium.r -
                                            organism_color_low.r);
                                        uint8_t g =
                                        organism_color_low.g + t *
                                            (organism_color_medium.g -
                                            organism_color_low.g);
                                        uint8_t b =
                                        organism_color_low.b + t *
                                            (organism_color_medium.b -
                                            organism_color_low.b);
                                        cell_colors[y][x] = sf::Color(r, g, b);
                                }
                        } else {
                                float nutrient_ratio = nutrients[y][x] /
                                    OrganismsConsts::P_MAX;
                                uint8_t intensity = static_cast<uint8_t>(50 +
                                    nutrient_ratio * 205);
                                cell_colors[y][x] = sf::Color(0, intensity, 0);
                        }
                }
        }
}

void Organism::handleKeyPress(const sf::Event::KeyPressed& key_event) {
        BaseCellAutomaton::handleKeyPress(key_event);

        switch (key_event.scancode) {
                case sf::Keyboard::Scan::R:
                reset();
                break;
                case sf::Keyboard::Scan::V:
                toggleVision();
                break;
                default:
                break;
        }
}

void Organism::reset() {
        initGrid();
        tick = 0;
        is_running = false;
        updateCellColors();
        cout << "Organism Simulation reset" << endl;
}

void Organism::toggleVision() {
        vision = !vision;
        cout << "vision " << vision << endl;
}

unsigned int Organism::getPopulationCount() const {
        unsigned count = 0;
        for (unsigned y = 0; y < num_cells; y++) {
                for (unsigned x = 0; x < num_cells; x++) {
                        if (grid[y][x] && organisms[y][x].is_alive) {
                                count++;
```

```
                        }
                }
        }
        return count;
}
```

grid.h

```
#pragma once

#include <SFML/Graphics.hpp>
#include <functional>
#include <iostream>
#include <memory>
#include <vector>

using namespace std;

class Grid {
        private:
        unsigned num_cells;
        unsigned cell_size;
        unsigned width;
        unsigned height;

        sf::Color grid_color;
        sf::VertexArray grid_lines;
        vector<vector<sf::RectangleShape>> cell_shapes;

        bool initialized = false;

        public:
        Grid(unsigned num_cells, unsigned cell_size, sf::Color grid_color);
        ~Grid();

        void init();

        unsigned getWidth() const { return width; }
        unsigned getHeight() const { return height; }
        unsigned getNumCells() const { return num_cells; }
        unsigned getCellSize() const { return cell_size; }
        void drawGrid(sf::RenderWindow& window);
        void drawCells(sf::RenderWindow& window, const vector<vector<sf::Color>>&
            cell_colors);
};
```

grid.cpp

```
#include "grid.h"

Grid::Grid(unsigned num_cells, unsigned cell_size, sf::Color grid_color)
: num_cells(num_cells), cell_size(cell_size), grid_color(grid_color) {
        width = cell_size * num_cells;
        height = cell_size * num_cells;
        cout << "Grid␣init␣" << endl;
}

Grid::~Grid() { cout << "Grid␣destroyed" << endl; }

void Grid::init() {
        if (initialized) {
                return;
        }

        grid_lines.setPrimitiveType(sf::PrimitiveType::Lines);
        for (unsigned x = 0; x <= num_cells; x++) {
                float x_pos = static_cast<float>(x * cell_size);
                grid_lines.append({{x_pos, 0}, grid_color});
                grid_lines.append({{x_pos, static_cast<float>(height)}, grid_color});
        }
        for (unsigned y = 0; y <= num_cells; y++) {
```

```
                float y_pos = static_cast<float>(y * cell_size);
                grid_lines.append({{0, y_pos}, grid_color});
                grid_lines.append({{static_cast<float>(width), y_pos}, grid_color});
        }

        cell_shapes.resize(num_cells, vector<sf::RectangleShape>(num_cells));
        for (unsigned y = 0; y < num_cells; y++) {
                for (unsigned x = 0; x < num_cells; x++) {
                        cell_shapes[y][x].setSize(
                        sf::Vector2f(static_cast<float>(cell_size - 1),
                            static_cast<float>(cell_size - 1)));
                        cell_shapes[y][x].setPosition(
                        sf::Vector2f(static_cast<float>(x * cell_size),
                            static_cast<float>(y * cell_size)));
                }
        }

        initialized = true;
}

void Grid::drawGrid(sf::RenderWindow& window) { window.draw(grid_lines); }

void Grid::drawCells(sf::RenderWindow& window, const vector<vector<sf::Color>>&
    cell_colors) {
        for (unsigned y = 0; y < num_cells; y++) {
                for (unsigned x = 0; x < num_cells; x++) {
                        cell_shapes[y][x].setFillColor(cell_colors[y][x]);
                        window.draw(cell_shapes[y][x]);
                }
        }
}
```

consts.h

```
#pragma once

#include <SFML/Graphics.hpp>
#include <string>
#include <tuple>
#include <vector>

using namespace std;

namespace Consts {
        const sf::Color GRID_COLOR(100, 150, 255);
        const sf::Color BACKGROUND_COLOR(0, 0, 0);

        constexpr float DEFAULT_UPDATE_INTERVAL = 0.15f;
        constexpr unsigned FRAME_LIMIT = 60;
        constexpr float SPEED_STEP = 0.02f;
        constexpr float MIN_SPEED = 0.5f;
        constexpr float MAX_SPEED = 0.02f;
}  // namespace Consts

namespace OrganismsConsts {
        constexpr unsigned DEFAULT_NUM_CELLS = 256;
        constexpr unsigned DEFAULT_CELL_SIZE = 4;
        constexpr unsigned L = 15;
        constexpr unsigned T = 3;
        constexpr float P_MAX = 10.0f;
        constexpr float P1 = 35.0f;
        constexpr unsigned A = 30;
        constexpr float R = 1.0f;
        constexpr float DELTA_P = 5.0f;
        constexpr float DELTA_E = 2.0f;
        constexpr float DELTA_R = 3.0f;

        const sf::Color ORGANISM_COLOR_HIGH(255, 0, 0);
        const sf::Color ORGANISM_COLOR_MEDIUM(255, 200, 0);
        const sf::Color ORGANISM_COLOR_LOW(255, 100, 0);

        const sf::Color NUTRIENT_HIGH(0, 255, 0);
```

```
        const sf::Color NUTRIENT_MEDIUM(0, 200, 0);
        const sf::Color NUTRIENT_LOW(0, 100, 0);
}  // namespace OrganismsConsts

namespace NeuralNetworkConsts {
        constexpr unsigned DEFAULT_NUM_CELLS = 256;
        constexpr unsigned DEFAULT_CELL_SIZE = 4;

        const sf::Color CELL_COLOR_ACTIVE(255, 0, 0);    // red
        const sf::Color CELL_COLOR_REST(0, 255, 0);      // green
        const sf::Color CELL_COLOR_RECOVERY(0, 0, 255);  // blue

        constexpr float A = 0.3f;
        constexpr float P = 3.0f;
        constexpr int T = 5;
        constexpr int B = 8;
        constexpr int SOURCE_PERIOD = 15;
}  // namespace NeuralNetworkConsts

                namespace LiveGameConsts {
        constexpr unsigned DEFAULT_NUM_CELLS = 32;
        constexpr unsigned DEFAULT_CELL_SIZE = 20;

        const sf::Color CELL_COLOR(255, 0, 0);

        inline const vector<tuple<string, string, unsigned>> CONWAY_PATTERNS = {
                {"random", 0},
                {"block", 1},
                {"beehive", 2},
                {"loaf", 3},
                {"blinker",4},
                {"toad", 5},
                {"glider", 6},
                {"pulsar", 7},
                {"gosper_glider_gun", 8}};

        inline const vector<tuple<string, string, unsigned>> B2S012_PATTERNS = {
                {"random", 0},
                {"background_with_vertical", 1},
                {"background_with_eye", 2},
                {"background_with_eight", 3},
                {"background_with_nail", 4},
                {"background_with_gates", 5}};
}  // namespace LiveGameConsts
```